Theoretical Computer Science

Distributional Learning of Context-Free Languages under Fixed Finite-Semigroup Typing

--Manuscript Draft--

Manuscript Number:	TCS-D-26-00494R1
Article Type:	Regular Paper (10 - 40 pages)
Section/Category:	Algorithms, automata, complexity and games
Keywords:	Grammatical inference; Context-free languages; Distributional learning; Positive-data identification; Finite semigroups
Corresponding Author:	Takayuki Kuriyama Independent Researcher
First Author:	Takayuki Kuriyama
Order of Authors:	Takayuki Kuriyama
Abstract:	We study positive-data learning of context-free languages whose nonempty factors are typed by a fixed morphism s: Sigma+ -> S into a finite semigroup. A polynomial-time set-driven operator reconstructs each substitutable target from finite positive witnesses; a separate conservative learner identifies nonempty targets in Gold's sense. The encoded characteristic-data size is polynomial in grammar size and a typed shortest-yield parameter. Ordinary grammar thickness suffices when the typing is determined by a fixed prefix–suffix window and a finite-group value; for linear targets, linear grammar size alone suffices. Conversely, a fixed two-element semilattice typing yields finite targets with grammars of size and ordinary thickness both O(n) that require exponentially long characteristic samples. The lower bound applies to any set-driven operator admitting characteristic samples for the fixed-typing class. Retyping closure, examples beyond fixed windows, and a Dyck-language obstruction delineate the framework's scope.

Powered by Editorial Manager® and ProduXion Manager® from Aries Systems Corporation

Response to Reviewers — TCS-D-26-00494

Manuscript: "Distributional Learning of Context-Free Languages under Fixed Finite-Semigroup Typing"

Author: Takayuki Kuriyama

We thank the Editor and both reviewers for their detailed and constructive reports. Their comments led us to correct the learning algorithm, remove an unused typing component, repair the terminal-rule completeness argument, and substantially reorganize the manuscript.

Unqualified section and statement numbers refer to the revised manuscript; submitted numbers are marked "submitted" or "former". Subheadings such as "Section 7" or "Sections 8–10" follow the reviewers' grouping and refer to the submitted manuscript. We number the Editor's comment E.1 and the reviewers' comments R1.1–R1.37 and R2.1–R2.11; quotations are excerpts with typographical normalization, omissions marked by ellipses, and related remarks grouped by topic. The locator guide below refers to the additions-marked revised manuscript and is checked against the marked PDF. Because the revision substantially reorganizes the paper, the marked PDF displays extensive additions, including passages rewritten for presentation rather than newly claimed results. An unmarked LaTeX source file is supplied separately.

Overview of changes

Framework of the revised manuscript. Reviewer quotations retain the reviewers' original notation. The revised manuscript fixes a surjective finite-semigroup typing $s : \Sigma^+ \to \mathsf{S}$, with the fresh-identity completion $\widehat{s}$ distinguishing empty yields. This is more than a change of terminology. In the submitted Definition 2.3, the relation induced by $h : \Sigma^* \to M$ was tested on all $x, y \in \Sigma^*$, including λ and positive words of type $h(\lambda)$. The revised definition uses the positive restriction $s = h|_{\Sigma^+}$ and tests only nonempty factors, while $\widehat{s}$ handles empty yields separately (Lemma 2.2). Thus each submitted fixed-h target class is contained in the corresponding revised fixed-s class; we do not claim the two definitions are equivalent. The title has been changed accordingly. Section 5 now types arbitrary CFG productions directly, including unit, nullable, and mixed right-hand sides, without start-symbol separation. BUNF (Definition 6.1) is retained only for the quantitative bounds in Sections 6–8. The revised sequential learner Learn_s is conservative; no incremental polynomial-data bound is claimed.

Central corrections. The revision makes four central corrections. First, it separates the set-driven reconstruction operator Rec_s from the conservative sequential learner Learn_s, so that the Gold learner now stabilizes syntactically. Second, the unused outer-context type annotations are removed, and the occurrence-indexed hypothesis symbols are quotiented to one substring-indexed symbol $[\![x]\!]$ per observed factor. The resulting batch construction follows Yoshinaka's substring grammar (as also presented by Clark), replacing the window comparison in the unary rules by equality of s-types. Third, the terminal-rule completeness gap is repaired: witnesses for typed terminal rules, already implicit in the submitted characteristic sample, are now listed explicitly, and the completeness proof uses unary Rule (U) before lexical Rule (L) when necessary. Fourth, fixed-s substitutability is stated for nonempty internal fragments, while λ is distinguished in typed grammars by the fresh identity of $\mathsf{S}^\sharp$.

Scope and prior work. The revised Introduction cites the author's 2014 preprint (arXiv:1409.6247v3) as historical background and acknowledges Yoshinaka's (2015) general disjoint-sort framework, which also supplies positive-data identification, polynomial-time hypothesis updates, and polynomial-cardinality characteristic data under the corresponding many-sorted specialization. These general learning guarantees and fixed-window signatures are not presented as independent novelties. The semigroup identities and recognizable-language background of Proposition 3.5 are attributed to Pin's published 1986 monograph; the positive-image/kernel formulation is proved explicitly in the revised manuscript. Yoshinaka (2008, Theorem 1) already provided polynomial encoded-size characteristic-data bounds for fixed-window classes. The revised paper's quantitative contribution instead concerns encoded-length bounds beyond fixed windows in the stated window–group and linear regimes, together with a fixed-typing exponential lower bound despite small ordinary grammar thickness. We also give expressive separations and obstructions. The elementary retyping-closure facts and classical fixed-window criteria are recorded to delimit the framework, rather than claimed as independent new closure or semigroup theorems. The relevant prior work is cited in the manuscript and addressed in the responses below.

Narrowed claims. Presentation-order independence and polynomial characteristic-data bounds are claimed only for the set-driven operator Rec_s, the latter only in the stated quantitative regimes; the sequential learner Learn_s is proved to have Gold convergence and polynomial-time updates, without an incremental polynomial-data guarantee (Section 2.1; R1.1). The submitted Section 7 title is withdrawn accordingly (R2.3). The former claim that outer-context types are necessary is withdrawn, and the quotient construction needs no separate context-transport rule (R2.2).

New quantitative results. Proposition 6.4 gives finite Clark–Eyraud substitutable languages L_n with grammars of size and ordinary thickness $O(n)$: under trivial typing Rec_{s_0} has polynomial characteristic data, whereas under a fixed two-element semilattice typing every characteristic sample for Rec_{s_U} contains a word of length $3 \cdot 2^n - 2$. By Proposition 6.4(iv), the same holds for any set-driven operator admitting characteristic samples for both L_n and $L_n \setminus \{w_n^*\}$ in the fixed-typing class. Corollary 6.6 gives $\tau_{s_U}^{\mathrm{typ}}(G) \geq 3 \cdot 2^n - 2$ for every reduced BUNF grammar G of L_n, regardless of size, and Lemma 7.6, an elementary semigroup fact recorded for completeness, shows $U_1 \notin \mathbf{LI} \vee \mathbf{Grp}$; hence the algebraic condition of Corollary 7.5 cannot simply be dropped from a uniform characteristic-data bound. Neither necessity for every typing nor any lower bound on the running time or convergence stage of arbitrary Gold learners is claimed (R2.1).

Reorganization and clarifications. The manuscript has been substantially shortened (R1.2). Definitions are consolidated in Section 2; the reconstruction operator precedes the target-side refinement used for completeness; Sections 6–8 reorganize the quantitative discussion and answer Reviewer 2's question on the linear theorem (R2.3); the three former boundary sections form Section 9, which also contains Proposition 9.7 suggested by Reviewer 1; normalization proofs are in Appendices A–B; and Appendix C proves Proposition 6.4(i) by a reversible deterministic parser. The closure facts of the submitted Appendix A are condensed in Proposition 3.2 (R1.36). For the normalized linear grammar G_0, Theorem 8.3 is made explicit as $\|\mathsf{Wit}(\widetilde{G}_0)\| = O(|G_0|^3 |\mathsf{S}^\sharp|^4)$; the same degree is not claimed relative to an unnormalized input grammar. We also explicitly attribute the known fixed-window closure and nonclosure properties to Yoshinaka (2008, Proposition 1) (Section 3), clarify the factorization $\mathsf{S}^n = \mathsf{S}E(\mathsf{S})\mathsf{S}$ (Proposition 3.5), and clarify the essential-node width estimate in the proof of Lemma 7.1 (Appendix A.1). Changes to Proposition 9.1 and the sharpness of $B \geq 2$ are described in R1.33–R1.35.

Selected renumbering guide. Because the manuscript was substantially reorganized, the following table collects the submitted items most frequently referred to in the reports.

Submitted manuscript	Revised manuscript
Definition 2.3 ($\sim_s$)	Definition 2.1
Definition 3.2 (Clark–Eyraud)	unnumbered special case, Section 3.1
Proposition 3.5	Proposition 3.3
Theorem 3.7	Theorem 3.4
Example 4.3	removed
Submitted Rules (2), (3)	quotient construction; unary Rule (U)
Lemma 5.2(i)	witness set (2), Theorem 5.2
Theorem 5.7	Theorem 5.3
Corollary 5.8	Corollary 5.4
Proposition 7.2	Proposition 8.1
Theorem 7.9	Theorem 8.3
Theorem 8.6	Theorem 9.2 (simplified proof)
Corollary 8.7	removed
Propositions 10.3–10.5	Proposition 9.1; criterion after Proposition 3.3
Lemma 9.11	Lemma 9.6
Appendix A (closure facts)	Proposition 3.2 (adapted)
CCL_p, CCL	CTR_B, CTR
Counter alphabet $d, u, ;$	$d \mapsto \uparrow$ (increment), $u \mapsto \downarrow$ (decrement), $; \mapsto \#$ (reset)
L_Y (classical language)	L_{Luk} (Łukasiewicz)
Props. 9.2, 9.6, 9.9; Lemma 9.8	brief statements/citations in Section 9.3

Locator guide for the additions-marked revised manuscript. Each location below is anchored to a named section or statement, with its actual PDF page and a margin-line number checked in the freshly compiled additions-marked manuscript. A section anchor may cover multiple revised paragraphs; the named statement identifies the relevant paragraph.

Item	Revised-manuscript locator
E.1	Definition 2.1 (marked PDF p. 4, margin l. 161); Sections 4–5 (marked PDF p. 8, margin l. 322)
R1.1	Section 2.1 (marked PDF p. 3, margin l. 121); Section 4.2 (marked PDF p. 9, margin l. 359); Section 5.2 (marked PDF p. 10, margin l. 408)
R1.2	Introduction (marked PDF p. 1, margin l. 19); Appendices A–C (marked PDF p. 22, margin l. 852)
R1.3	Introduction and related work (marked PDF p. 1, margin l. 19)
R1.4	Introduction (marked PDF p. 1, margin l. 19); Definition 2.1 (marked PDF p. 4, margin l. 161)
R1.5	Section 1.2 (marked PDF p. 2, margin l. 66)
R1.6	Section 9.3 (marked PDF p. 20, margin l. 783)

Item	Revised-manuscript locator
R1.7	Section 4.2 (marked PDF p. 9, margin l. 359)
R1.8	Section 2 (marked PDF p. 3, margin l. 96); Section 4.1 (marked PDF p. 8, margin l. 323)
R1.9	Section 5.1 (marked PDF p. 10, margin l. 374)
R1.10	Introduction (marked PDF p. 1, margin l. 19); Section 3 (marked PDF p. 5, margin l. 189)
R1.11	Lemma 2.2 (marked PDF p. 4, margin l. 163); Section 3.1 (marked PDF p. 6, margin l. 239)
R1.12	Section 3.1 (marked PDF p. 6, margin l. 239); Proposition 3.3 (marked PDF p. 6, margin l. 250); Proposition 3.5 (marked PDF p. 8, margin l. 303)
R1.13	Section 2 (marked PDF p. 3, margin l. 96); Theorem 3.4 (marked PDF p. 7, margin l. 275)
R1.14	Theorem 3.4 (marked PDF p. 7, margin l. 275)
R1.15	Section 2 (marked PDF p. 3, margin l. 96); Section 5.1 (marked PDF p. 10, margin l. 374)
R1.16	Section 2 (marked PDF p. 3, margin l. 96)
R1.17	Section 5.1 (marked PDF p. 10, margin l. 374)
R1.18	Section 4 (marked PDF p. 8, margin l. 322); Sections 5.1–5.2 (marked PDF p. 10, margin l. 374)
R1.19	Proposition 5.1 (marked PDF p. 10, margin l. 390)
R1.20	Section 5.1 (marked PDF p. 10, margin l. 374)
R1.21	Section 5.1 (marked PDF p. 10, margin l. 374)
R1.22	Proposition 5.1 (marked PDF p. 10, margin l. 390)
R1.23	Proposition 5.1 (marked PDF p. 10, margin l. 390)
R1.24	Section 4.2 (marked PDF p. 9, margin l. 359); Section 5.2 (marked PDF p. 10, margin l. 408)
R1.25	Section 4.2 (marked PDF p. 9, margin l. 359); Section 5.2 (marked PDF p. 10, margin l. 408)
R1.26	Section 4.1 (marked PDF p. 8, margin l. 323)
R1.27	Theorem 4.2 (marked PDF p. 9, margin l. 346)
R1.28	Theorem 5.2 (marked PDF p. 10, margin l. 409)
R1.29	Theorem 5.3 (marked PDF p. 11, margin l. 429)
R1.30	Theorem 5.5 (marked PDF p. 11, margin l. 448)
R1.31	Section 8 (marked PDF p. 16, margin l. 639); Appendix B (marked PDF p. 23, margin l. 906)
R1.32	Section 8 (marked PDF p. 16, margin l. 639); Appendices A–B (marked PDF p. 22, margin l. 852)
R1.33	Theorem 9.2 and Section 9.2 (marked PDF p. 19, margin l. 775)
R1.34	Section 9 (marked PDF p. 18, margin l. 722)
R1.35	Proposition 9.1 (marked PDF p. 19, margin l. 748); Sections 9.2–9.3 (marked PDF p. 19, margin l. 767)
R1.36	Lemma 9.6 (marked PDF p. 20, margin l. 802)
R1.37	Proposition 9.7 (marked PDF p. 21, margin l. 817)
R2.1	Introduction (marked PDF p. 1, margin l. 19); Section 3 (marked PDF p. 5, margin l. 189); Proposition 6.4 (marked PDF p. 13, margin l. 511); Lemma 7.6 (marked PDF p. 16, margin l. 623); Sections 8–9.2 (marked PDF p. 16, margin l. 639)
R2.2	Section 4.1 (marked PDF p. 8, margin l. 323); Sections 5.1–5.2 (marked PDF p. 10, margin l. 374)
R2.3	Theorem 8.3 (marked PDF p. 17, margin l. 662); Proposition 8.4 (marked PDF p. 17, margin l. 681)
R2.4	Sections 4–5 (marked PDF p. 8, margin l. 322); Appendices A–B (marked PDF p. 22, margin l. 852)
R2.5	Proposition 3.1 (marked PDF p. 5, margin l. 195)
R2.6	Section 5.1 (marked PDF p. 10, margin l. 374)
R2.7	Section 5.1 (marked PDF p. 10, margin l. 374)
R2.8	Section 5.1 (marked PDF p. 10, margin l. 374); Theorem 5.2 (marked PDF p. 10, margin l. 409)
R2.9	Theorems 5.2–5.3 (marked PDF p. 10, margin l. 409)
R2.10	Corollary 5.4 (marked PDF p. 11, margin l. 434)
R2.11	Proposition 8.1 (marked PDF p. 16, margin l. 643); Lemma 8.2 (marked PDF p. 16, margin l. 650); Appendix B (marked PDF p. 23, margin l. 906)

Response to the Editor

E.1. Editor (quoted excerpt): "The reviewers agree that the scientific contribution has merit. However, they both point to many issues with the presentation, many of which leak also into the scientific part of the paper and are not merely language revisions. This is especially relevant in the learning part of the paper. ... Specifically, any correctness issues should be carefully considered."

Response: We agree. The submitted learner has been replaced by a conservative Gold learner; the terminal-rule gap in the completeness proof has been repaired, and the terminal-rule witnesses are now listed explicitly; the unused outer-context annotations have been removed. Fixed-s substitutability is now stated for nonempty internal fragments, with λ handled separately. Section 4 defines the batch operator and sequential learner before Section 5 introduces target-side yield typing for completeness. Polynomial characteristic data are attributed to the set-driven batch operator, while the sequential learner is asserted to have Gold stabilization with polynomial-time updates.

Revision location: Definition 2.1; Section 4.2; Section 5.1 and Theorem 5.2; Corollary 5.4; Sections 6–8.

Reviewer 1

General assessment and learning algorithm

R1.1. Comment (quoted excerpt): "The main result of this submission is an elegant generalization of the results of Clark and Eyraud (2007) on the learning of substitutable context-free languages (SCFLs) and of Yoshinaka (2008) on the learning of k,l-SCFLs. ... The only serious error is in the definition of their learning algorithm, but this is easily repairable by following prior work."

Response: We thank the reviewer for identifying the central correctness problem. We agree that the qualitative identification scheme and many of the arguments are standard or follow earlier work. The revised Introduction therefore identifies these guarantees as specializations of Yoshinaka's 2015 framework, while separately crediting the 2008 fixed-window results; we do not present the qualitative learning theorem or polynomial-time hypothesis updates as independent contributions. Recomputing the batch grammar after every datum need not stabilize syntactically. The revision therefore separates a set-driven operator Rec_s (Section 4.1) from a conservative sequential learner Learn_s (Section 4.2), using the same conservative rebuild-on-missed-positive-example pattern as Clark–Eyraud's SGL and Yoshinaka's (k, ℓ)-SGL. Corollary 5.4 proves eventual syntactic stabilization. As noted by Eyraud–Heinz–Yoshinaka (2016, Appendix), now cited in Section 4.2, the correct rebuild may be delayed until the first positive example missed after a characteristic sample has appeared. Hence Learn_s is claimed to have Gold convergence with polynomial-time updates, but not the incremental polynomial-time-and-data criterion (ibid., Definition 2.5). Our polynomial characteristic-data guarantees are set-based (ibid., Definition 2.8): they concern Rec_s, only in the stated quantitative regimes, not uniformly for all s. Section 2.1 now makes this distinction explicit.

Revision location: Section 2.1; Section 4.2; Corollary 5.4; Theorem 7.2; Theorem 8.3 and the following clarification.

R1.2. Comment (quoted excerpt): "On the other hand, the presentation leaves considerable room for improvement. By addressing the absence of definitions for technical terms, unnecessarily long proofs, inconsistent use of metavariables, and the over-fragmentation of the paper into too many sections, the authors could substantially reduce the burden on the reader, and I recommend a major revision along these lines. ... I suspect that this material could be presented in less than half the current length."

Response: We have reduced the paper from 40 to 26 pages (including the appendices), while maintaining the central proofs and the technical definitions requested by the reviewers. The revised sections consolidate the reconstruction and convergence arguments, the polynomial-data bounds, and the expressive examples. Binary–unit–nullable normal form (BUNF) is defined in Definition 6.1 only as a device for the quantitative estimates. The qualitative witness construction and completeness proof of Section 5 work directly for arbitrary CFG right-hand sides. The quantitative normalization and reversible stack-parser arguments are in Appendices A–C. We have also addressed the reviewer's four presentation concerns: technical terminology is defined at first use or in Section 2; proof-local detail was shortened or moved to Appendices A–C; word metavariables in the Section 9 concatenation identity and quotient argument no longer reuse the typing symbols s, t, and the temporary component count in the completeness proof of Theorem 5.2 has been renamed from h to ν. The integer offset in Section 9 is denoted by δ, typed productions in Section 5 use ρ, and the capped-counter bound in Section 9 uses B; the former boundary sections have been consolidated into Section 9. Sections 6–8 remain separate because they concern distinct quantitative regimes (general typings, window–group typings, and linear targets). We acknowledge that the revised paper is shorter but not half the length of the submitted version; it also incorporates the substantive corrections and new results requested by the reviewers.

Revision location: Sections 1–2; Sections 4–9; Appendices A–C.

Section 1

R1.3. Comment (quoted excerpt): "Stating the paper's place within the field and the significance of the language families considered would make the paper more compelling. Conversely, the current text is too heavily weighted toward technical content and could be made more concise."

Response: Agreed. Section 1.2 now begins with a concise account of the fixed-s batch reconstruction operator and conservative Gold learner as the learning foundation, expressly acknowledging that the learning and polynomial-time guarantees specialize Yoshinaka's general many-sorted signature framework. Rather than listing those existing learning guarantees as separate novel contributions, the Introduction emphasizes two main contributions: (1) quantitative characteristic-data results, including their dependence on the chosen typing, the locally trivial–group and linear bounds, and the exponential lower bound; and (2) expressive scope and limitations, including specific examples beyond fixed windows, counter/Dyck obstructions, and comparison with Clark's congruential family. Elementary retyping closure and the classical fixed-window criterion are discussed after the list as background, not counted as new contributions. The fixed morphism is presented as a finite typing

bias (R2.1). The historical account distinguishes the author's 2014 preprint from Yoshinaka's subsequent general many-sorted signature learning theorem and identifies which results are particular to fixed finite-semigroup typing (see also R2.1).

Revision location: Section 1.

R1.4. Comment (quoted excerpt): "The notations $\mathcal{D}_L$ and $\mathcal{C}_h^{\mathrm{cf}}$ are not standard and require definitions. In particular, the phrasing 'the general context-free class $\mathcal{C}_h^{\mathrm{cf}}$' is liable to be misunderstood."

Response: Agreed. $\mathcal{D}_L(x)$ is defined at its first occurrence in the Introduction and again formally in the preliminaries. The notation $\mathcal{C}_s^{\mathrm{cf}}$ is now defined at its first use in the Introduction as the class of context-free $\sim_s$-substitutable languages. The unrestricted fixed-typing class RS_s is then introduced before $\mathsf{RS} = \bigcup_s \mathsf{RS}_s$. Section 3 develops these classes and $\mathcal{C}_s^{\mathrm{lin}}$ formally. The phrase "general context-free class" has been removed.

Revision location: Introduction; Section 2.2 and the opening of Section 3.

R1.5. Comment (quoted excerpt): "The contributions are enumerated as (1)-(6), but it is unnecessary to count technical details as separate contributions. Moreover, at this point the text uses many terms whose technical meaning has not yet been made clear, and readers may not be able to follow them."

Response: Agreed. Instead of the submitted six-item list, Section 1.2 now gives a short unnumbered paragraph on the qualitative learning construction and its relation to Yoshinaka's general framework, followed by only the two contributions described in R1.3; proof-local devices are not counted. The quantitative item summarizes polynomial upper bounds and exponential lower bounds on characteristic data without developing proof-local machinery in the Introduction. The typed shortest-yield parameters and binary–unit–nullable normal form (BUNF) are formally defined in Section 6. The second item instead focuses on expressive examples and obstructions; the classical fixed-window criterion and elementary closure facts are mentioned separately after the list as background. The term *locally trivial* is defined in Section 2.2; the join $\mathbf{LI} \vee \mathbf{Grp}$ is explained in Section 1.2 and formalized in Section 7. The Clark–Eyraud (2007, Section 6.1) example $L = \{a, aa\}$, recalled after the fixed-s definition in Section 1.1, shows that trivial typing fails (two factors share a context but have different distributions); parity typing then distinguishes them. It illustrates the definition and is not a separation from fixed-window classes.

Revision location: Section 1.1 (minimal example after the definition of fixed-s substitutability); Section 1.2.

R1.6. Comment (quoted excerpt): " 'Yoshinaka's classical language' is often called 'the Łukasiewicz language' in the literature."

Response: Agreed. We use "Łukasiewicz language," add the Autebert–Berstel–Boasson (1997) handbook reference, and state its relation to the one-bracket Dyck language concisely. We also now cite Yoshinaka (2008, Section 3) for the prior observation that this language is outside every fixed (k, ℓ) class, and distinguish our stronger conclusion that it lies outside the finite-typing union RS.

Revision location: Section 9.3.

Section 2

R1.7. Comment (quoted excerpt): "I doubt the claim that 'the learners considered in this paper produce their output independently of the order in which positive examples are observed.' It appears that the learning algorithm is being conflated with the grammar-construction algorithm that it uses as a subroutine. Indeed, the 'learning algorithm' in Section 5 does not converge, and hence does not satisfy Gold's criterion. Even if the learning algorithm were corrected to follow Clark & Eyraud, the output would still differ, when learning, e.g., the language $a^n c b^n$, between the case where the characteristic sample itself is received first and the case where the first positive example received is very long."

Response: We withdrew the claim that the sequential learner is order-independent. It is the batch reconstruction operator Rec_s, not the conservative learner Learn_s, that is set-driven. Section 4.2 restores the explicit argument: for any infinite target, take w longer than every word in the sample at the final rebuild on a convergent text. The limiting grammar lacks $[\![w]\!]$, but prepending w yields another text whose rebuilt and limiting grammar contains $[\![w]\!]$. Both limit hypotheses generate the same language. See also Clark (2013, Section 4, Example 1).

Revision location: Section 2.1; Section 4.2; Corollary 5.4.

R1.8. Comment (quoted excerpt): "This section should thoroughly define the necessary notation and terminology, including basic notions such as (linear) context-free grammars - some terms appear to be defined ad hoc in footnotes. Those terms include 'type', 'reconstruction', 'realize', 'reachable', 'observation', 'control', 'local configuration', etc. If these words carry no mathematical definition required to read the paper and were introduced only to aid intuition, then in the present form they tend to confuse the reader, and in some cases it would be better not to name them at all. A flood of excessive terminology can itself burden the reader and impede smooth understanding."

Response: Agreed. Section 2 now consolidates the grammar and learning notions needed later, including linear CFGs, reachable/productive/reduced grammars, the positive-data model, set-driven reconstruction, characteristic samples, distributions, and the fixed-s classes. Proof-local labels without mathematical content have been removed or are introduced only at their point of use (e.g., the former *finite typed reconstruction basis*; R1.9). Section 4.1 uses the standard substring-indexed factor nonterminals $[\![x]\!]$, defined at first use. The revision also defines $|w|_a$ in Section 2, spells out SGL (*Substitution Graph Learner*) and (k,ℓ)-SGL ((k,ℓ)-*Substitutable Grammar Learner*) at their point of use, specifies $\mathsf{Acc} \subseteq \mathsf{S}^\sharp$ at its first appearance, and removes the unused class notation $[x]_{\equiv_L}$. As for the listed terms: *reachable* and yield *type* are defined in the preliminaries and typing setup, respectively; *reconstruction* refers to the defined batch operator Rec_s; *observation* is not a separate technical notion, Section 4.1 speaking only of observed factors and their hypothesis symbols; *realize* and *local configuration* have been removed; and *control* remains only in standard uses such as control sets and pushdown finite control.

Revision location: Section 2; Section 4.1; Section 5.1; Appendix A.2.

R1.9. Comment (quoted excerpt): "The term 'finite typed reconstruction basis', a combination of undefined words, is clearly defined here; however, if it denotes an atomic concept for which none of its four constituent words is individually defined, a shorter term would be easier to understand."

Response: Agreed. Rather than rename this proof-local object, we removed both the term and the basis itself. The revised completeness argument uses only the yield-typed target grammar and the finite witness set $\mathsf{Wit}(\widetilde{G})$ that is actually needed for exact reconstruction.

Revision location: Section 5.1.

Section 3

R1.10. Comment (quoted excerpt): "The paper emphasizes (somewhat excessively and awkwardly) that h is fixed. To match this, RS_h should be defined before RS. The family RS is not shown to be learnable from positive examples (and indeed it is not learnable); what is learnable is RS_h."

Response: Agreed. The Introduction now explicitly defines RS_s before $\mathsf{RS} := \bigcup_s \mathsf{RS}_s$, and the Gold-identification theorem covers every *nonempty* language in $\mathcal{C}_s^{\mathrm{cf}} = \mathsf{RS}_s \cap \mathrm{CFL}$ for fixed s, not the whole class RS_s; no learnability claim is made for the varying-s union RS, which is used only for expressive comparison. The revision also states that neither RS nor $\mathsf{RS} \cap \mathrm{CFL}$ is identifiable from positive data on a nonempty alphabet, since these classes contain all finite languages and Σ^*.

Revision location: Abstract; Introduction; Section 2.2 and the opening of Section 3.

R1.11. Comment (quoted excerpt): "Definition 3.2 requires care. On this point, the way Clark & Eyraud (2007) state their definition is somewhat careless; however, given that in Section 6.1 of that paper they state that a^+ is substitutable, and that their grammar-construction algorithm excludes the empty string, it is reasonable to interpret the intent of their definition as Yoshinaka's (2008) 0,0-substitutability. . . . "

Response: We agree with the reviewer's interpretation of Clark–Eyraud (2007): with nonempty internal factors, their intended case coincides with Yoshinaka's $(0,0)$ case (Section 3.1). As suggested, we cite Clark (2013), distinguish Clark–Eyraud's Definition 5 (general strings) from this nonempty-fragment convention, restrict fixed-s substitutability to nonempty internal fragments, and give λ a separate fresh-identity type in the general CFG refinement, including nullable non-start nonterminals. New Lemma 2.2 proves that freely adjoining a fresh identity gives an equivalent all-fragment typing formulation, without identifying λ with nonempty words of a pre-existing identity type of the positive-image semigroup. The hypothesis reconstruction is unchanged; its target-side completeness proof now treats an arbitrary rule $A \to u_0 B_1 u_1 \cdots B_r u_r$ directly, with typed child yields and empty yields distinguished by the fresh-identity extension $\widehat{s}$. BUNF (Definition 6.1) is used only for the quantitative estimates. Section 3.1 explains the issue for $L = a^+$: (a,λ) belongs to both $\mathcal{D}_L(\lambda)$ and $\mathcal{D}_L(a)$, whereas (λ,λ) belongs only to $\mathcal{D}_L(a)$, so comparing the empty factor would exclude a^+. The center-marked language $L_{\mathrm{c}} = \{a^n c b^n : n \geq 0\}$ is used consistently as an illustration; no argument concerning it compares an empty internal factor. The (k,ℓ) definition now uses u_1, u_2, v_1, v_2 for outer contexts, p, q for the fixed windows, and r_1, r_2 for the variable interiors of the compared fragments pr_1q and pr_2q.

Revision location: Definition 2.1; Lemma 2.2; the Clark–Eyraud special case in Section 3.1; the fixed-window definition in Section 3.1.

R1.12. Comment (quoted excerpt): "Proposition 3.5. $\sqcup$ and pref_k are undefined. . . . The proof is very long. If the multiplication on T (as a monoid) were given explicitly in advance, then one line would suffice to explain that τ is a monoid homomorphism. Rephrasing in terms of $h_{k,\ell}$ or $M_{k,\ell}$ is needlessly cumbersome."

Response: We agree that the fixed-window argument was too long and used unnecessary notation. The symbol $\sqcup$ has been removed, and prefix and suffix notation, including the short-word convention, is defined before use. Proposition 3.3 (submitted Proposition 3.5) now defines fixed windows using a quotient finite semigroup $W_{k,\ell}$ of Σ^+, rather than a separate four-case multiplication table. The short/long coding is a two-sided

congruence, giving an associative quotient product. The same positive-word typing $s_{k,\ell}$ is used in Proposition 3.5, Lemma 7.1, Theorem 7.2, and Lemma 7.4. Short words, on which (k, ℓ)-substitutability imposes no comparison, remain individually typed. The fixed-window counterexample criterion following Proposition 3.3 summarizes the separation argument used later, and the classical locally trivial-semigroup criterion, the window–finite-group factorization, and the resulting thick-data bound are stated separately in Proposition 3.5, Lemma 7.4, and Corollary 7.5.

Revision location: Proposition 3.3 and the fixed-window counterexample criterion; Proposition 3.5; Lemma 7.4; Corollary 7.5.

R1.13. Comment (quoted excerpt): "Rather than saying that h is given as data (the meaning is not crystal clear), it would be cleaner to state early in the paper something like 'fix an efficiently computable $h : \Sigma^* \to M$'. Define 'linear context-free'. ... 'Explicit finite monoid homomorphism' is not defined. ... Technically unnecessary words should be removed, especially in the statements of theorems."

Response: Agreed. We fix a surjective finite-semigroup morphism $s : \Sigma^+ \to \mathsf{S}$ by its multiplication table and letter images, and compute its values on nonempty words in linear time. Linear context-free grammars are defined in the preliminaries and the main theorem assumes a fixed efficiently computable s.

Revision location: Sections 2–3.2.

R1.14. Comment (quoted excerpt): "I do not understand 'The proof for fixed h'. Do you mean 'The proof for Theorem 3.7'?"

Response: Agreed. This stray phrase has been deleted; the revised text states the relevant theorem and its proof directly.

Revision location: Section 3.2.

Section 4

R1.15. Comment (quoted excerpt): "Definition 4.2. 'productive' and 'reachable' are undefined. The phrasing 'This section proceeds in the order typed local control → finite typed basis.' is too casual."

Response: Agreed. These notions are defined in the preliminaries, and the typed grammar is defined as the reduced part of the full refinement, so repeated qualifications are unnecessary. The quoted informal sentence has been removed.

Revision location: Section 2; Section 5.1.

R1.16. Comment (quoted excerpt): "Footnote 2: 'productive' is already used earlier. Define it in Section 2 rather than in a footnote."

Response: Agreed. Reachable and productive are now defined in Section 2 before the target refinement is introduced, and the former explanatory footnote has been removed.

Revision location: Section 2; Section 5.1.

R1.17. Comment (quoted excerpt): "The text says 'the set of reachable and productive typed nonterminals'; is there any nonterminal of $\widetilde{G}$ that fails this condition? This may be intended as a helpful reminder or emphasis, but it confuses the reader instead."

Response: The reviewer is correct: every symbol retained in $\widetilde{G}$ is reachable and productive because $\widetilde{G}$ is defined as the reduced part of the full yield-typed refinement. The repeated qualification has therefore been removed.

Revision location: Section 5.1.

R1.18. Comment (quoted excerpt): "The first paragraph states no reason why 'different occurrences must be treated as different typed copies' at all. Expressions making clear 'for what purpose and for what reason' are needed. Example 4.3 is labeled '(why typing is needed)', yet why it is needed is not stated. ... If the authors wish to explain the reason, the development would be smoother if they first presented the learning algorithm and then, within its completeness proof, stated: 'here we assume the target grammar is type-refined, and we show that the learner's hypothesis grammar can mimic it.' "

Response: We have adopted the reviewer's suggested order of presentation. Section 4 first defines the batch reconstruction operator Rec_s and the conservative sequential learner Learn_s, with no target refinement assumed. Section 5 then introduces the yield-typed target grammar specifically for the completeness proof and states explicitly that the purpose is to show that the already-defined hypothesis grammar can mimic typed target derivations. Outer-context typing has been removed (R2.2). The remaining yield typing supports the unary Rule (U) step of that simulation for arbitrary right-hand sides. For a typed production $\rho : X \to u_0 Y_1 u_1 \cdots Y_r u_r$, its canonical yield $t_\rho = u_0 \omega(Y_1) u_1 \cdots \omega(Y_r) u_r$ satisfies $\widehat{s}(\omega(X)) = \widehat{s}(t_\rho)$. When both yields are nonempty, their common s-type and an observed shared context license Rule (U); Rule (B) and Rule (L) then handle the right-hand-side components. Empty-type yields contribute only λ and are treated separately.

Revision location: Section 4; Section 5.1 and Theorem 5.2.

R1.19. Comment (quoted excerpt): "Lemma 4.4. It is confusing that an expression concerning derivations in G appears immediately after 'In the full typed refinement $\widetilde{G}^{\text{full}}$,'. It is unnecessary or should be moved to the end of the sentence."

Response: The former Lemma 4.4 has been removed in the reorganization. Its needed content is now folded into Proposition 5.1, which states language preservation and the yield invariant directly and keeps derivations of the original grammar and of the typed refinement syntactically separate.

Revision location: Proposition 5.1.

R1.20. Comment (quoted excerpt): "Lemma 4.8. 'If X is reachable and productive in $\widetilde{G}$,' This likewise confuses the reader."

Response: Resolved by the reduced-refinement convention of R1.17: the former Lemma 4.8 is deleted together with this redundant hypothesis.

Revision location: Section 5.1.

R1.21. Comment (quoted excerpt): "The opening paragraph (the definitions of ω and χ) would connect better if placed immediately before the definition of CS."

"Since the term 'characteristic sample' is defined in Def. 2.1, avoid writing 'Define the characteristic sample by' here unless it is immediately clear that this set $CS(G)$ satisfies Def. 2.1."

"The term 'realizable' is defined here but used earlier, whereas the term 'realized', used in many places, is undefined."

"The phrase 'canonical observation word' could be defined when $CS(\widetilde{G})$ is defined."

Response: Agreed. The canonical yield $\omega(X)$ is defined after the trimmed refinement, before first use, and the canonical context $\chi(X)$ immediately before the finite witness set. $\mathsf{Wit}(\widetilde{G})$ is called a witness set until exact reconstruction has been proved, and only then identified as a characteristic sample. The unused basis, $NT(\cdot)$, Rule$(\cdot)$, and the realized/realizable and exhibited terminology have been removed.

Revision location: Section 5.1.

R1.22. Comment (quoted excerpt): "Lemma 4.6. This is far too trivial a corollary. Either label it as a Corollary or delete it."

"Lemma 4.7. Given what the reader has understood so far, this is trivial and needs no proof. . . . "

"Theorem 4.9. I cannot understand what this theorem shows. . . . "

"Theorem 4.12. The first sentence should be removed. The second and third sentences are not understandable owing to the undefined term 'exhibited'."

Response: The former Lemmas 4.6–4.7 and Theorems 4.9 and 4.12 have been removed. The typed-refinement development now consists of a general-CFG type lifting invariant (Proposition 5.1) and a general right-hand-side witness construction. The BUNF definition occurs only in the quantitative Section 6.

Revision location: Section 5.1.

R1.23. Comment (quoted excerpt): "α is a string, not a symbol."

"Said 'In particular, if $X \to YZ$,' but the antecedent of the 'if' is unrelated to its consequent."

"Lemma 4.5. 'For (iii), every derivation in G erases typed annotations' I cannot understand this. Do you mean 'For (iii), every derivation in G with typed annotations erased gives . . . '?"

"Is $NT(\widetilde{G})$ not the same as $\widetilde{V}$ on page 9?"

"Definition 4.11. Is Rule$(\widetilde{G})$ different from $\widetilde{P}$ on page 9? . . . "

Response: Agreed. These sentences disappear in the rewrite, and $NT(\widetilde{G})$ and Rule$(\widetilde{G})$ are replaced by $\widetilde{V}$ and $\widetilde{P}$. The lifting statement is now Proposition 5.1 (R1.19); its proof says that erasing type annotations maps a typed derivation to a derivation of the original grammar, which addresses the question on former Lemma 4.5(iii).

Revision location: Section 5.1.

Section 5

R1.24. Comment (quoted excerpt): "The learning algorithm as defined here does not converge. This is the fatal error of the paper."

Response: We agree. The submitted learning algorithm did not converge, and this was a central correctness error. As detailed in R1.1, we replaced it with a conservative sequential learner and proved syntactic stabilization in Corollary 5.4.

Revision location: Section 4.2; Corollary 5.4.

R1.25. Comment (quoted excerpt): "Corollary 5.8. This cannot be accepted as a proof. By the definition in Section 5.1, the learner's hypothesis grammar keeps growing as it receives more positive examples."

Response: See R1.1 and R1.24. We changed the learner rather than only the proof; Corollary 5.4 is the resulting stabilization statement.

Revision location: Section 4.2; Corollary 5.4.

R1.26. Comment (quoted excerpt): "Remark 5.1. I cannot grasp the point. What does 'soundness of a rule' mean? ... If one sentence is to be added, something like 'the grammar is designed so that the language derived by each nonterminal $[x : u, v]$ should be the set of strings substitutable with x' might aid intuition. Stating a mathematical property concisely helps the reader more than deploying adjectives that carry no mathematical definition."

"Lemmas 5.3 and 5.4 merely repeat definitions and should be deleted."

Response: The remark and definition-restating lemmas are deleted. The hypothesis now has one symbol $[\![x]\!]$ per observed nonempty factor rather than one per occurrence context (R2.2), and, in the spirit of the reviewer's suggestion, the mathematical property is stated directly: whenever $[\![x]\!] \Rightarrow^* w$, we have $x \equiv_L w$ and $s(w) = s(x)$ (R1.27). We do not claim that a finite-sample nonterminal generates the entire syntactic-congruence class.

Revision location: Section 4 (Rules (B), (U), (L), (S) and the soundness invariant).

R1.27. Comment (quoted excerpt): "Lemma 5.2. In the second sentence, the first claim is trivial and the second is not understandable. What is the corresponding local configuration? Its correspondence with the part following 'More precisely' is unclear."

Response: Agreed. The former Lemma 5.2 and the undefined "corresponding local configuration" terminology have been removed. Their role is replaced in the proof of Theorem 4.2 by the simpler invariant

$$[\![x]\!] \Rightarrow^* w \implies x \equiv_L w \text{ and } s(w) = s(x),$$

proved by induction on derivation-tree height. The unary case uses one shared sample context and fixed-s substitutability; the branching case uses that $\equiv_L$ is a monoid congruence. Start rules then give hypothesis soundness.

Revision location: Theorem 4.2 and its displayed soundness invariant.

R1.28. Comment (quoted excerpt): "Page 14, line 1: 'Since G is in SSBNF' $\to \widetilde{G}$?"

Response: Corrected during the rewrite; the relevant grammar is named explicitly throughout the completeness proof.

Revision location: Theorem 5.2.

R1.29. Comment (quoted excerpt): "Theorem 5.7. Make the statement mathematical. What do you mean by 'a finite sample exposes the finite typed reconstruction basis'? A sample is a string set and the FTRB is a tuple."

Response: Theorem 5.3 now states the exact implication $\mathsf{Wit}(\widetilde{G}) \subseteq K \subseteq L \Rightarrow L(\mathsf{Rec}_s(K)) = L$ with all assumptions in the theorem statement.

Revision location: Theorem 5.3.

Section 6

R1.30. Comment (quoted excerpt): "If the paper adopts the stance of Remark 6.4, then the fact that the grammar construction can be carried out in polynomial time is almost trivial, so the content of this section could be presented in a single paragraph."

Response: The former standalone complexity section is now the short Section 5.3, essentially one counting argument (Theorem 5.5): with $n_K = \sum_{w \in K}(|w| + 1)$, which also counts λ, there are $O(n_K^2)$ factor nonterminals and $O(n_K^3)$ branching and unary candidates, giving an explicit $O(n_K^4)$ worst-case construction time. The update-time consequence is stated in prose rather than as a separate corollary.

Revision location: Section 5.3.

Section 7

R1.31. Comment (quoted excerpt): "This is a special case of the arguments in the preceding sections and could be much shorter ... Too long explanation may blur the focus — what the reader should understand, and what differs from the preceding sections."

"Lemma 7.3. 'strict linear derivation' is not defined."

Response: We substantially shortened the material corresponding to submitted Section 7, now Section 8. The main text retains only Proposition 8.1 and Lemma 8.2, which provide the single-spine form and short-yield bound used by Theorem 8.3; the short proof of Lemma 8.2 follows its statement in Section 8, while the normalization proof, now written with uniform spine chains, is in Appendix B. The undefined term *strict linear derivation* has been removed.

Revision location: Proposition 8.1 and the surrounding discussion; Lemma 8.2; Appendix B.

R1.32. Comment (quoted excerpt): "Proposition 7.2 does not need two full pages on the proof. I do not know whether exactly the same restriction has been defined before, but it is a combination of existing restrictions that are known to be inessential, and most readers will accept Proposition 7.2 as trivial or as folklore."

"Lemmas 7.5 and 7.6 are too trivial."

Response: Agreed. The former two-page normalization proof is replaced by the arrangement described in R1.31 (Proposition 8.1 with its proof in Appendix B, and Lemma 8.2 with a short proof); the former Lemmas 7.5 and 7.6 are absorbed into those arguments or deleted. In line with the folklore remark, Appendix A.2 cites standard terminal-isolation and binarization references (Autebert–Berstel–Boasson, 1997; Hopcroft–Motwani–Ullman, 2007), while Appendix B verifies the one-wrapper-child spine structure with polynomial grammar size and construction time.

Revision location: Proposition 8.1 and the surrounding discussion; Lemma 8.2; Theorem 8.3 and the following clarification; Appendices A.2 and B.

Sections 8–10

R1.33. Comment (quoted excerpt): "The choice of the alphabet $\{d, u, ;\}$ is unfortunate. The semicolon in particular is very hard to read within the running text, and u is a source of confusion because it almost always denotes a string elsewhere in the paper. . . . "

"The proof of Theorem 8.6 should be simplified. Introducing complex formulas and lavishing Greek letters in an attempt to shorten the witness is not the essence of the theorem and only burdens the reader. . . . "

Response: Agreed. We renamed $d \mapsto \uparrow$ (increment), $u \mapsto \downarrow$ (decrement), $; \mapsto \#$ (reset), and p as B, and adopted the reviewer's suggested witness in its essential form, for which we are grateful: Theorem 9.2 now uses $(\uparrow\downarrow)^k \uparrow^{B-1} \#^\ell$ and $(\uparrow\downarrow)^k \uparrow^B \#^\ell$, which share the empty context and the same fixed-window boundaries, while a preceding $\uparrow$ distinguishes their distributions. The criterion following Proposition 3.3 avoids an expanded six-variable argument. A footnote justifies the sharpness of $B \geq 2$ by showing that CTR_1 is $(1, 1)$-substitutable.

Revision location: Section 9.2.

R1.34. Comment (quoted excerpt): "There is no need to devote an entire subsection to this material. It is unclear what Corollary 8.7 establishes. . . . Proposition 3.5 and Theorem 8.6 are meaningful; but folding them into Corollary 8.7 actually obscures the paper's important contributions."

" 'We note that a block of a word in CCL refers to each of the subwords delimited by the separator symbol ;.' This is a new definition, not a note. In fact, the proof of Theorem 8.6 uses 'block' of a CCL string in an entirely different sense."

"Sections 8-10 all concern the richness of the class of the learning target languages; consolidating them into a single section and keeping the explanations and proofs concise would help the reader grasp the overall picture."

Response: We agree that a union of separately learnable families need not itself be learnable. The former Corollary 8.7 has therefore been deleted: RS is used only for expressive comparison, whereas learning is asserted for fixed-s context-free slices, and submitted Proposition 3.5 and Theorem 8.6 remain separate statements (Proposition 3.3 and Theorem 9.2). The former three boundary sections are consolidated in Section 9, with concise proofs. The overloaded *block* terminology is replaced by explicit factors and height-based descriptions, and the witnessing typing $s_B = h_B|_{\Sigma_c^+}$ of CTR_B is the positive restriction of the DFA transition-monoid morphism h_B in Section 9.2, where Yoshinaka's earlier regular separation (2008, Proposition 1) is credited. The submitted Section 10 example $\Delta^* = (\{a^n b^n : n \geq 0\})^*$ is now the shorter Proposition 9.1 (see R1.35). The Kleene-star nonclosure example based on $\{a^n b a^n : n \geq 0\}$ and the appendix on CCL_1 have been deleted.

Revision location: Section 9.

R1.35. Comment (quoted excerpt): "Definition 9.1. Define the symbol #, and make its use consistent in Section 9."

"Proposition 9.2 is trivial and requires no proof. Proposition 9.6 is also a well-known fact, as already stated in Ginsburg & Greibach, Deterministic context-free languages, Information and Control 9 (1966), 620-648."

"Lemma 9.8. The relationship between the Łukasiewicz language and the Dyck language is also well known."

"The proof of Proposition 10.3 may also be omitted or greatly shortened. In the proof of Proposition 10.5, consider aligning the symbols with those of Definition 3.3."

Response: The reset symbol # is defined once and used consistently. The deterministic context-freeness of the uncapped counter CTR (former Proposition 9.2) is stated briefly via a deterministic pushdown counter with reset. For D_1 (former Proposition 9.6) and the Łukasiewicz language, we cite Ginsburg–Greibach (1966); the identity $L_{\mathrm{Luk}} = D_1 b$ (former Lemma 9.8) is cited to Autebert–Berstel–Boasson (1997) (see also R1.6). The proof of submitted Proposition 10.3 is condensed in Proposition 9.1: deterministic context-freeness follows from a short pushdown argument, and substitutability is proved using the boundary-compatible integer-offset sets $\mathcal{Q}(x)$ and the explicit two-sided concatenation criterion ($\star\star$). The former Proposition 10.5 proof is replaced by the criterion following Proposition 3.3, and the capped-counter regularity argument is now one elementary DFA paragraph.

Revision location: Section 9.

R1.36. Comment (quoted excerpt): "Lemma 9.11 is a fundamental property of $\sim_h$ substitutability; since the basic mathematical properties of $\sim_h$ substitutability are collected in an appendix, placing it there is one option."

Response: The submitted appendix of basic properties is no longer retained: its intersection and inverse-homomorphism facts are condensed in Proposition 3.2, with erased factors distinguished under the nonempty-fragment convention. The remaining right-quotient property is the two-line Lemma 9.6, used immediately afterwards, so we keep it in place rather than create an appendix for one fact.

Revision location: Lemma 9.6.

Section 11

R1.37. Comment (quoted excerpt): "It is worth mentioning that $\mathsf{RS} \cap \mathrm{CFL}$ is a subset of the class of Clark's congruential CFLs."

Response: Added as Proposition 9.7. Clark (2010) places substitutable and (k, ℓ)-substitutable languages in the congruential family; the revised text also explicitly identifies his Dyck example. The proposition extends the containment slice-wise to every fixed s, hence to $\mathsf{RS}(\Gamma) \cap \mathrm{CFL}$. Properness over $\{a, b\}$ is witnessed by D_1, which is congruential but lies outside RS.

Revision location: Proposition 9.7.

Reviewer 2

For the learning-algorithm correction, see R1.1 and R1.24; for the terminal-rule completeness repair raised below, see R2.8. We also recognize the reviewer's overall concern that the learning contribution and motivation for the fixed-typing setting were not sufficiently clear in the submitted version; R2.1 now separates inherited learning results from the new quantitative and expressive contributions.

General positioning of fixed-h

R2.1. Comment (quoted excerpt): "At a general level, the paper offers an abstraction of the learning of Yoshinaka's (k, ℓ)-substitutable languages by replacing the fixed-length window by an arbitrary recognizable congruence, and lifts the known results accordingly. The abstraction places the problem in a powerful and well-understood framework, but it requires a whole morphism to be known in advance for each language, whereas the earlier settings fixed a single uniform parameter valid across the entire class. The paper acknowledges that learning the morphism is outside its scope, yet the interest of the setting – beyond providing a more abstract framework for the study of (k, ℓ)-substitutable languages – remains unclear. A promising direction would be to deepen the links with pushdown automata, which Sections 8 to 10 already touch on. The links with the work of Takada ... are also worth investigating. In a more practical direction, this work could be read as extending automaton typing ... to context-free substitutable languages, and the conditions a typing must satisfy could be studied."

Response: We agree that supplying a finite-semigroup morphism is a stronger prior bias than fixing (k, ℓ), and we do not infer this morphism. However, s is fixed uniformly across the entire target class $\mathcal{C}_s^{\mathrm{cf}}$; it is not chosen anew for each individual target language. For $s = s_{k,\ell}$, our reconstruction specializes to Yoshinaka's 2008 substring-rule construction, up to the stated representation choices.

The interest of fixing s is to make the quantitative cost of a typing bias explicit, not to learn the bias. For every fixed s, nonempty targets in $\mathcal{C}_s^{\mathrm{cf}}$ are identifiable from positive data, whereas the encoded size of characteristic data can depend sharply on the algebra of s. The locally trivial–group condition supplies a sufficient polynomial regime; a fixed two-element semilattice typing already exhibits an exponential obstruction. We do not claim a characterization of all polynomial regimes.

The qualitative identification guarantees and polynomial-time hypothesis updates are specializations of Yoshinaka's 2015 general many-sorted framework, not independent novelties. Beyond the fixed-window encoded-size bound already obtained by Yoshinaka (2008, Theorem 1), the new content is quantitative and expressive. First, we obtain characteristic-data encoded-length bounds using ordinary thickness for window–finite-group typings (Corollary 7.5), and a grammar-size-only bound for linear targets (Theorem 8.3). Second, a fixed two-element semilattice typing yields finite targets with grammar size and ordinary thickness $O(n)$ but requiring a characteristic-sample word of length $3 \cdot 2^n - 2$ for every set-driven operator admitting characteristic samples for the specified distinguishing pair (Proposition 6.4); Corollary 6.6 gives the corresponding intrinsic typed-thickness obstruction. This contrasts with examples whose ordinary thickness is already exponential. Third, nonregular linear and nonlinear examples lie outside every fixed-window class (Propositions 8.4 and 9.1), while the counter/Dyck results limit what fixed finite typing can represent (Section 9). These results do not provide a method to infer s, a uniform bound for all fixed typings, or a convergence-stage lower bound for arbitrary Gold learners.

We have also clarified the algebraic scope. Yoshinaka (2008, Proposition 1) established closure of the unrestricted fixed-(k, ℓ)-substitutable class under intersection of two class members and under nonerasing inverse homomorphisms, as well as nonclosure under arbitrary regular intersection and arbitrary inverse homomorphisms; its context-free subclass need not be closed under intersection. Proposition 3.2 records elementary finite-typing analogues, including regular filtering and erasing inverse images after changing the typing; these closure facts are not claimed as new theorems. Section 3 also describes adequate typing as finite-index congruence separation of conflicting distributions, not a procedure for finding the typing. Finally, the revised Introduction cites automata typing bias (Coste et al., 2004), and Section 8 gives a limited connection with Takada's regular control sets for linear grammars. We claim neither a general pushdown characterization nor an equivalence with Takada's learning framework.

Revision location: Introduction; Section 3 (concise Yoshinaka comparison immediately before Proposition 3.2); Section 4; Propositions 3.2 and 3.5; Main Theorem 3.4(iv); Proposition 6.4, Corollary 6.6, and Appendix C; Section 7, Lemmas 7.4 and 7.6, Corollary 7.5, Appendix A.1; Proposition 8.4; Section 9.2 (L_{Yreg} definition); the control-set discussion in Section 8.

Main concern 1: outer context typing and transport

R2.2. Comment (quoted excerpt): "My main concern is the typed refinement, and specifically transport. Rule (2) allows a nonterminal to move between arbitrary contexts with no condition whatever on their types. Is this intended? If the outer components m, n record where a nonterminal may appear, why does the rule that relocates it ignore them entirely? As far as I can tell the components are then read by no proof: Lemma 4.6 is invoked only for the yield component, and replacing $A_p^{m,n}$ by A_p throughout leaves every result intact. Does the author agree, and if not, where exactly is an outer type consulted? The situation seems to me to follow from Definition 2.3 itself, whose algebraic condition bears on the substituted fragment and imposes nothing on the surrounding context – so that a learner mirroring that definition has nothing to guard transport with. If that is right, then the typed transport announced in Section 1.1 does not occur anywhere in the article, and the claim that occurrences must be separated according to their outer types cannot be established by any example. The yield typing, which is genuinely essential and is what becomes vacuous under the trivial monoid, should not be left in a footnote."

Response: We agree, and the observation permits a stronger simplification than merely deleting the outer type annotations. No proof in the *submitted version* consulted the outer components, so $A_p^{m,n}$ has been replaced by the yield-typed A_p on the target side. On the hypothesis side, the context index is also inessential: submitted Rule (2) made all symbols $[x : u, v]$ with the same factor x mutually derivable. Quotienting those symbols therefore gives one nonterminal $[\![x]\!]$ per observed factor. Submitted Rule (3) then becomes exactly the unary rule $[\![x]\!] \to [\![y]\!]$ when x and y share an observed context and $s(x) = s(y)$. The separate transport rule disappears. Accordingly, we withdraw the submitted claim that occurrences must be separated by their outer types, as well as the typed-transport claim in submitted Section 1.1.

Apart from the start-symbol presentation, the treatment of λ, and redundant identity rules, the revised batch grammar follows Yoshinaka's substring-indexed rule scheme (2008, Section 4), also presented by Clark (2013, Section 4). For window typing $s_{k,\ell}$ its nontrivial unary rules coincide with Yoshinaka's; for general fixed s they require type equality and a shared sample context. Soundness rests on the invariant of Theorem 4.2, $[\![x]\!] \Rightarrow^* w$ implies $x \equiv_L w$ and $s(w) = s(x)$ (R1.27), which consults no outer type.

Yield typing, which the reviewer rightly identifies as essential, is now discussed in the main text of Section 5.1 rather than in a footnote. It is needed only for the present witness-based completeness argument, where it licenses the unary steps from a canonical parent yield to a typed terminal yield or to a canonical right-hand-side yield. This is a requirement of the witness-based completeness proof, not a claim of algorithm-independent necessity.

Revision location: Section 4; Section 5.1 and Theorem 5.2.

Main concern 2: extent of the linear theorem

R2.3. Comment (quoted excerpt): "The linear result raises a different question: how large is the class it covers? The three closing sections situate RS, not $\mathcal{C}_h^{\mathrm{lin}}$, so the reader is given no sense of what Theorem 7.9 reaches beyond the linear substitutable languages. The capped-counter languages are regular and hence linear, so the separation from Yoshinaka's hierarchy already holds inside a linear class, whereas $\{a^n b^n\}^*$ is not linear, so the final section establishes nothing at that level. As it stands the linear theorem reads as a technical strengthening on a class of unknown extent, the non-regular linear case being left open."

Response: Section 8 now provides both a concrete nonregular linear example and a structural closure mechanism. Proposition 8.4 proves that $L_{\pm,e}$ is linear, nonregular, and substitutable under a fixed finite-semigroup typing, but belongs neither to the Clark–Eyraud substitutable class nor to any (k,ℓ)-substitutable class. More generally, Proposition 3.2(ii), together with closure of linear languages under regular intersection, gives the implication used in the proof of Proposition 8.4:

$$L \in \mathcal{C}_s^{\mathrm{lin}}, \quad R_{\mathrm{reg}} = g^{-1}(\mathsf{Acc}_g) \quad \Longrightarrow \quad L \cap R_{\mathrm{reg}} \in \mathcal{C}^{\mathrm{lin}}_{s\times(g|_{\Sigma^+})}.$$

Here $t = g|_{\Sigma^+}$, with codomain $g(\Sigma^+)$, as in Proposition 3.2(ii). For a suitable acceptance set $\mathsf{Acc}_t \subseteq g(\Sigma^+)^\sharp$, $g^{-1}(\mathsf{Acc}_g) = \widehat{t}^{-1}(\mathsf{Acc}_t)$: the fresh identity records whether λ is accepted. The revised proof now displays $\mathsf{T} = g(\Sigma^+)$, $t = g|_{\Sigma^+}$ and $R_{\mathrm{reg}} = \widehat{t}^{-1}(\mathsf{Acc}_t)$ explicitly. That proof writes $L_{\pm,e} = L_{\mathrm{all}} \cap R_{\mathrm{reg}}$ with $L_{\mathrm{all}} = \{a^n \xi b^n : n \geq 0,\ \xi \in \{c,d,e\}\}$, which is linear and Clark–Eyraud substitutable under the nonempty-factor convention, and $R_{\mathrm{reg}} = (aa)^*c(bb)^* \cup a(aa)^*d\,b(bb)^* \cup a^*eb^*$; a subsequent paragraph exhibits a more economical four-element typing with a short verification. The regular-level separation was already known from Yoshinaka (2008, Proposition 1); the capped-counter family is retained only as the finite-state companion to the uncapped counter CTR. No separation from Takada's control-set approach is claimed.

We withdraw the submitted Section 7 title, *The Linear Subclass: A Full Polynomial Time-and-Data Theorem*, as a characterization of the sequential learner. Theorem 8.3 now establishes polynomial characteristic data only for the set-driven operator Rec_s, relative to a chosen linear CFG representation of a fixed-s substitutable target, with no incremental polynomial-data guarantee for Learn_s. This does not contradict de la Higuera's negative result for the unrestricted class of linear languages: our bound applies only to $\mathcal{C}_s^{\mathrm{lin}}$ for fixed s.

Revision location: Proposition 3.2(ii); Theorem 8.3 and its scope clarification; Proposition 8.4 (regular-filter argument in its proof) and the following four-element typing paragraph.

Further comments

R2.4. Comment (quoted excerpt): "The paper would gain considerably from being simplified: much of Section 4 is definitional, several results restate what their proofs have just established, and objects are regularly used before they are introduced, which forces the reader to hold material in suspense. Relatedly, since $\tilde{G}$ is by construction the trimmed refinement of $\tilde{G}_{\mathrm{full}}$, its symbols are reachable and productive by definition; repeating this qualification at every occurrence adds length without adding content, and one sentence recording the convention would suffice."

Response: We agree. The repeated reachable/productive qualifications have been removed and the reduced-refinement convention is stated once in Section 5.1. Redundant results, including former Lemmas 4.6–4.7 and 5.3–5.4 and Theorems 4.9 and 4.12, are deleted (R1.22, R2.9). To avoid forward use, the batch reconstruction operator is defined before the target-side refinement, ω before its first use, χ immediately before the witness set, and *locally trivial* in Section 2.2 before Proposition 3.5. Section 2.1 also explicitly forwards readers to the definitions of Rec_s and Learn_s in Sections 4.1 and 4.2. The quantitative material is separated into Sections 6 (general typed thickness), 7 (window–finite-group typings), and 8 (linear targets), with long normalization proofs in the appendices.

Revision location: Section 2.1; Section 4; Section 5.1; Sections 6–8; Appendices A–B.

R2.5. Comment (quoted excerpt): "Proposition 3.1: the hypothesis $L = h^{-1}(F)$ characterises the regular languages exactly. Replace 'In particular' by 'Thus' (and the proof could make explicit why)."

Response: Proposition 3.1 states the standard regular-language recognition characterization in the finite-semigroup formulation, using the canonical fresh-identity extension $\widehat{s}$ to distinguish λ. If $s(x) = s(y)$ for positive x, y, then $\widehat{s}(uxv) = \widehat{s}(uyv)$ for every outer context. Thus any language recognized by $\widehat{s}$ is $\sim_s$-substitutable, establishing the required consequence. We cite Eilenberg (1974, Chapter III) for the classical result.

Revision location: Proposition 3.1.

R2.6. Comment (quoted excerpt): "Definition 4.11: the basis $B(\widetilde{G})$ is never used. Is it useful to introduce it as well as Theorem 4.12 and Remark 4.13? (Notations NT() and Rule() are unnecessary since it is sufficient to define $G = ()$ and use defined V and P.)"

Response: Agreed. These objects and notations have been removed.

Revision location: Section 5.1.

R2.7. Comment (quoted excerpt): "Example 4.3 is not in SSBNF"

Response: The former example has been removed entirely; the revised proof no longer needs it.

Revision location: Section 5.1.

R2.8. Comment (quoted excerpt): "Lemma 5.2 (i): If X carries several terminal rules $\omega(X)$ is not sufficient, and rule (3) should be used (proof needs to be corrected accordingly)."

Response: The reviewer identified a genuine terminal-rule completeness gap. The revised Section 5 uses an arbitrary typed CFG production $\rho : X \to u_0 Y_1 u_1 \cdots Y_r u_r$. For each retained typed rule, the witness set contains $u_X t_\rho v_X$, where t_ρ replaces each typed nonterminal child by its canonical yield. In particular, a terminal rule $X \to a$ yields the witness $u_X a v_X$. Together with $u_X \omega(X) v_X$, this supplies the shared context and equal positive s-type needed for unary Rule (U); repeated Rules (B) and (L) then emit any terminal-only right-hand side. For general right-hand sides, Rule (B) splits the nonempty components and induction handles the positive-type children; children of fresh-identity type contribute only λ. Thus unit, nullable, and mixed productions are handled without imposing BUNF in the completeness proof (Theorem 5.2).

Revision location: Section 5.1 (witness set); Theorem 5.2 (terminal-rule base case).

R2.9. Comment (quoted excerpt): "Are Lemmas 5.3 and 5.4 needed?"

Response: Agreed. They are deleted.

Revision location: Sections 4–5.1.

R2.10. Comment (quoted excerpt): "Corollary 5.8 Should begin with 'For a fixed homomorphism h from Σ^* to a finite monoid M, ...', or 'Being given a homomorphism h from...' "

Response: Corollary 5.4 now begins with the analogous more general assumption: *For a fixed surjective semigroup homomorphism $s : \Sigma^+ \to \mathsf{S}$ to a finite semigroup.* As requested, the homomorphism is fixed uniformly across its target class.

Revision location: Corollary 5.4.

R2.11. Comment (quoted excerpt): "section 7.1 Transformation to standard SSLNF (3 pages) could be condensed or given in appendix."

Response: We took the appendix option: Section 8 retains only Proposition 8.1 and Lemma 8.2, used directly by Theorem 8.3, with the short proof of Lemma 8.2; the normalization proof is in Appendix B.

Revision location: Proposition 8.1 and the surrounding discussion; Lemma 8.2; Theorem 8.3 and the following clarification; Appendix B.

Click here to access/download
Revised manuscript (unmarked, editable source file)
TCS-D-26-00494_Manuscript_Clean.tex

Revised manuscript (marked up) Click here to view linked References

Distributional Learning of Context-Free Languages under Fixed ~~Finite-Monoid~~ Finite-Semigroup Typing

Takayuki Kuriyama
Independent Researcher, Tokyo, Japan
growup.kuriyama@gmail.com

Abstract

We study positive-data learning of context-free languages whose nonempty factors are typed by a fixed morphism $s : \Sigma^+ \to \mathsf{S}$ into a finite semigroup. A polynomial-time set-driven operator reconstructs each substitutable target from finite positive witnesses; a separate conservative learner identifies nonempty targets in Gold's sense. The encoded characteristic-data size is polynomial in grammar size and a typed shortest-yield parameter. Ordinary grammar thickness suffices when the typing is determined by a fixed prefix–suffix window and a finite-group value; for linear targets, linear grammar size alone suffices. Conversely, a fixed two-element semilattice typing yields finite targets with grammars of size and ordinary thickness both $O(n)$ that require exponentially long characteristic samples. The lower bound applies to any set-driven operator admitting characteristic samples for the fixed-typing class. Retyping closure, examples beyond fixed windows, and a Dyck-language obstruction delineate the framework's scope.

Keywords: Grammatical inference; Context-free languages; Distributional learning; Positive-data identification; Finite semigroups

1 Introduction

1.1 Background and motivation

Gold [1] established limitations of positive-data identification. Distributional learning yields positive results for restricted context-free classes, notably Clark–Eyraud substitutability [2] and Yoshinaka's (k, ℓ)-substitutability [3]. For background see de la Higuera [4] ; Kanazawa's work on learning categorial grammars offers a complementary approach [5, 6].

Relation to earlier work. The author's 2014 preprint [7] introduced substitutability relative to recognizable equivalence relations, preceding the publication of Yoshinaka's general many-sorted signature framework [8]. For a fixed semigroup morphism $s : \Sigma^+ \to \mathsf{S}$, its nonempty fibers can be represented as disjoint sorts, equipped with typed concatenation and promotion to the whole-string sort. Under this specialization, Yoshinaka's general framework [8, Theorem 1 and Lemma 3] already supplies positive-data identification, polynomial-time hypothesis updates, and characteristic witnesses of polynomial cardinality in a typed target grammar. The particular choice of sorts is not attributed to that work.

The procedures below give an explicit fixed-typing construction, including a direct completeness proof for nullable and unit productions. Their existence, update-time bound, and finite-witness cardinality are not claimed as independent learning-theoretic novelties. Our principal new questions concern the *encoded length* of characteristic data, its polynomial upper and exponential lower bounds, and the algebraic and expressive consequences of finite typing. The locally trivial fixed-window criterion itself is classical.

We study a finite-semigroup-congruence version of this program. Let $s : \Sigma^+ \to \mathsf{S}$ be a homomorphism into a finite semigroup and $\sim_s := \ker s \subseteq \Sigma^+ \times \Sigma^+$. For $x \in \Sigma^*$, write $\mathcal{D}_L(x) := \{(u, v) : uxv \in L\}$. A language L is $\sim_s$-*substitutable* if , for nonempty x, y, $s(x) = s(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ imply $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Write $\mathcal{C}_s^{\mathrm{cf}}$ for the context-free $\sim_s$-substitutable languages, and call this the *fixed-s slice*. Thus full distributional equality is required only after a fixed finite algebraic comparison.

For a minimal example, take $\Sigma = \{a\}$ and $L = \{a, aa\}$. Although $(\lambda, \lambda) \in \mathcal{D}_L(a) \cap \mathcal{D}_L(aa)$, the context (a, λ) belongs to $\mathcal{D}_L(a)$ but not to $\mathcal{D}_L(aa)$. Thus L is not Clark–Eyraud substitutable [2, Section 6.1]. Under the parity morphism $s(a^n) = n \bmod 2$ into $\mathbb{Z}/2\mathbb{Z}$, however, a and aa have distinct types and $L \in \mathcal{C}_s^{\mathrm{cf}}$.

Finite-index semigroup congruences on Σ^+, equivalently kernels of finite-semigroup homomorphisms, provide finite typings. For each fixed s, write RS_s for the class of all $\sim_s$-substitutable languages, without a context-freeness restriction. For expressive comparison, let $\mathsf{RS} := \bigcup_s \mathsf{RS}_s$, where s ranges over finite-semigroup homomorphisms on Σ^+. Every regular language belongs to this union. Our learning results, however, concern only $\mathcal{C}_s^{\mathrm{cf}}$ for one fixed s; no learner is claimed for RS.

Unlike one fixed prefix–suffix window$^{\nearrow}$, s can combine independent finite-state factor features by a product. If the finite typing is allowed to be refined accordingly, the framework is stable under regular filtering and arbitrary inverse homomorphisms, including erasing ones (Proposition 3.2). This is a change-of-typing statement, not closure of one fixed-s slice. An algebraic criterion identifies the typings determined by fixed prefix–suffix windows on nonempty factors (Proposition 3.5); combining windows with finite groups yields a broader characteristic-data bound (Corollary 7.5). Examples below exhibit nonregular linear and nonlinear targets outside every fixed-window class, while counter and Dyck obstructions delimit the reach of fixed finite-semigroup typing.

The choice of s supplies a typing bias on terminal factors [9], not their occurrences. Its connection with regular control sets [10, 11] is confined to the linear setting of Section 8.

1.2 Contributions and main results

The finite-semigroup typing s is fixed as prior information for the whole target class $\mathcal{C}_s^{\mathrm{cf}}$; it is not inferred from positive data . We give a set-driven reconstruction operator Rec_s and a separate conservative Gold learner Learn_s. Their qualitative learning and polynomial-time guarantees specialize Yoshinaka's general framework [8], using the substring-indexed grammar construction of [3], and are not claimed as independent novelties. Our characteristic-data bounds concern Rec_s; no incremental polynomial-data guarantee is claimed for Learn_s. Our contributions are the following.

(1) *The quantitative cost of finite typing.* For arbitrary fixed s, typed shortest-yield lengths in a normalized grammar control the encoded size of characteristic data (Corollary 6.3). When s is determined by a fixed prefix–suffix window and a finite-group value, the size and ordinary shortest-yield thickness of the original reduced CFG suffice (Corollary 7.5). Equivalently, the positive image $s(\Sigma^+)$ may be any homomorphic image of a subsemigroup of a product of a finite locally trivial semigroup and a finite group (the class $\mathbf{LI} \vee \mathbf{Grp}$). For linear targets, the size of an arbitrary linear CFG alone suffices (Theorem 8.3). The fixed-window case already admits polynomial encoded-size bounds due to Yoshinaka [3, Theorem 1].

In contrast, a fixed two-element semilattice typing yields finite targets L_n with reduced grammars of size and ordinary thickness $O(n)$ for which every characteristic sample must contain a word w_n^* of length $3 \cdot 2^n - 2$. This holds for every set-driven operator admitting characteristic samples for both L_n and $L_n^- := L_n \setminus \{w_n^*\}$, which also belongs to the same

fixed-typing class (Proposition 6.4). The targets L_n admit polynomial characteristic data under trivial typing.

(2) *Expressive scope and limitations.* We construct nonregular linear (Proposition 8.4) and nonlinear (Proposition 9.1) targets outside every fixed-window class. We show that the uncapped counter language CTR and the one-bracket Dyck language D_1 lie outside RS (Section 9.3). We also place the context-free fixed-typing classes within Clark's congruential family (Proposition 9.7).

Elementary closure properties under changes of typing and the classical characterization of fixed-window typings are recorded in Section 3.

2 Preliminaries

Let Σ be a finite alphabet, let Σ^* be the free monoid with identity λ, and put $\Sigma^+ := \Sigma^* \setminus \{\lambda\}$. A pair $(u, v) \in \Sigma^* \times \Sigma^*$ is called a *terminal context*. For $L \subseteq \Sigma^*$ and $x \in \Sigma^*$, the *distribution* of x in L is $\mathcal{D}_L(x) := \{(u, v) \in \Sigma^* \times \Sigma^* : uxv \in L\}$. The *syntactic congruence* of L is defined by $x \equiv_L y$ iff $\mathcal{D}_L(x) = \mathcal{D}_L(y)$, for $x, y \in \Sigma^*$ [12, Chapter III, Proposition 10.2]. As usual, $\equiv_L$ is a monoid congruence: if $x \equiv_L x'$ and $y \equiv_L y'$, then $xy \equiv_L x'y'$.

For a word $w \in \Sigma^*$ and a letter $a \in \Sigma$, let $|w|_a$ denote the number of occurrences of a in w.

A context-free grammar (CFG) is a tuple $G = (V, \Sigma, P, S)$ with finite nonterminal set V, start symbol $S \in V$, and productions $A \to \alpha$ with $A \in V$ and $\alpha \in (V \cup \Sigma)^*$. A *sentential form* is any word in $(V \cup \Sigma)^*$. We write $\alpha \Rightarrow_G \beta$ for one derivation step in G, and $\Rightarrow_G^*$ for its reflexive-transitive closure. Thus $L(G) := \{w \in \Sigma^* : S \Rightarrow_G^* w\}$. For $A \in V$, put $L_G(A) := \{w \in \Sigma^* : A \Rightarrow_G^* w\}$. For a nonterminal $X \in V$, a terminal context (u, v) is a *terminal reaching context of X in G* if $S \Rightarrow_G^* uXv$. A nonterminal is *reachable* if it occurs in a sentential form reachable from S, *productive* if it derives a terminal word, *useless* if it is not both, and *nullable* if it derives λ; G is *reduced* if it has no useless nonterminal. A *unit production* (*unit rule*) is $A \to B$ with $A, B \in V$, and the *unit closure* is the reflexive-transitive closure of the induced relation on V. For $L(G) \neq \emptyset$, the *trim* (*reduced part*) of G deletes the nonproductive nonterminals and then the unreachable ones, each with its incident productions. A CFG is *linear* if every right-hand side contains at most one nonterminal. By *binarization* we mean replacement of longer right-hand sides by equivalent chains of binary productions using fresh nonterminals. We write CFL for the class of context-free languages. For any finite representation $\mathcal{R}$, $|\mathcal{R}|$ denotes the length of a fixed reasonable encoding; for a grammar this is polynomially equivalent to the usual symbol count of its productions. Following the thickness parameter used by Wakatsuki–Tomita [13] and adopted by Yoshinaka [3] (see also Eyraud–Heinz–Yoshinaka [14, Definition 2.11]), for a reduced CFG $G = (V, \Sigma, P, S)$ we use the shortest-yield *thickness*

$$\tau_G := \max_{A \in V} \min\{\, |w| : A \Rightarrow_G^* w,\ w \in \Sigma^* \,\}.$$

2.1 Learning from positive data

We use Gold's explanatory model [1]. Let H_0 denote a fixed CFG with one start symbol and no productions, so $L(H_0) = \emptyset$. We use texts only for nonempty targets and represent the empty language separately by H_0; equivalently one may adjoin the usual pause symbol. For a nonempty language L, a *text* is an infinite sequence $\mathcal{T} = (w_1, w_2, \ldots)$ over L in which every word of L occurs. Write $\mathcal{T}[n] = (w_1, \ldots, w_n)$ and $K_n := \{w_1, \ldots, w_n\}$. A learner Learn *identifies a nonempty L in the limit* if for every text $\mathcal{T}$ for L there are n_0 and a grammar H such that $\mathsf{Learn}(\mathcal{T}[n]) = H$ for every $n \geq n_0$ and $L(H) = L$. Thus the hypothesis representation itself eventually stabilizes. A sequential learner is *conservative* if it changes its current hypothesis only when the newly presented positive datum is not generated by that hypothesis.

We distinguish this sequential learner from a *set-driven reconstruction operator* Rec: this is a map from finite sets of words to grammars; in particular, its output is independent of presentation order and multiplicity. We call Rec *sample-consistent* if $K \subseteq L(\mathsf{Rec}(K))$ for every finite sample K . A finite set $\mathcal{F} \subseteq L$ is a *characteristic sample of L for Rec* if $\mathcal{F} \subseteq K \subseteq L$ implies $L(\mathsf{Rec}(K)) = L$ for every finite K; this is the positive-data, set-driven specialization of the characteristic samples of de la Higuera [15] (cf. [4, Section 7.3.3, Definition 7.3.3], formulated for sequential presentations). We measure sample size by $\|K\| := \sum_{w \in K}(|w| + 1)$, so that λ counts and $|K| \le \|K\|$, and call $\|\mathcal{F}\|$ the *characteristic data* of $\mathcal{F}$. Polynomial update time and polynomial characteristic data are distinct criteria [4, Sections 7.3.2–7.3.3]. In the terms of Eyraud–Heinz–Yoshinaka [14, Definitions 2.5 and 2.8], our polynomial characteristic-data guarantees are *set-based* (correct reconstruction by Rec_s, defined in Section 4.1, from every finite positive sample K satisfying $\mathcal{F} \subseteq K \subseteq L$, where $\mathcal{F}$ is a characteristic sample for L); the *incremental* criterion, a correct and stable hypothesis of Learn_s (defined in Section 4.2) as soon as such a sample has appeared, is not claimed. Gold stabilization of Learn_s is proved in Corollary 5.4.

2.2 Fixed finite-semigroup typing

Fix a finite semigroup S and a *surjective* semigroup homomorphism $s : \Sigma^+ \twoheadrightarrow \mathsf{S}$, represented by the multiplication table of S and the letter images $s(a)$ for $a \in \Sigma$. Thus $s(w)$ is computed by a left-to-right product in $O(|w|)$ time for nonempty w. Surjectivity is without loss of generality: an arbitrary finite semigroup typing may be replaced by its positive image. Whenever a later construction produces a non-surjective semigroup map (notably a product or inverse-image typing), its type codomain is implicitly restricted to the positive image before taking the fresh-identity completion. We work over a nonempty alphabet unless stated otherwise; for $\Sigma = \emptyset$, the only possible target languages are $\emptyset$ and $\{\lambda\}$, which the unchanged learner handles directly.

For a finite semigroup T, an element $e \in \mathsf{T}$ is *idempotent* if $e^2 = e$; write $E(\mathsf{T})$ for its idempotents. Call T *locally trivial* when $ete = e$ for all $e \in E(\mathsf{T})$ and $t \in \mathsf{T}$. Every nonempty finite semigroup has an idempotent. Consequently local triviality is preserved by surjective finite-semigroup morphisms: if $\pi : \mathsf{T} \to \mathsf{S}$ is surjective, every idempotent of S lifts to one in its preimage subsemigroup.

Definition 2.1 (fixed-s substitutability)**.** *A language $L \subseteq \Sigma^*$ is $\sim_s$-substitutable if for all $x, y \in \Sigma^+$, equality $s(x) = s(y)$ together with $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \varnothing$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$.*

Lemma 2.2 (canonical fresh-identity completion)**.** *Let $\mathsf{S}^\sharp = \mathsf{S} \cup \{\mathbf{1}_\sharp\}$ be the monoid obtained by adjoining a fresh identity $\mathbf{1}_\sharp$ to the finite semigroup S, even if S already has an identity. There is a unique monoid extension $\widehat{s} : \Sigma^* \twoheadrightarrow \mathsf{S}^\sharp$ of s, given by*

$$\widehat{s}(w) = \begin{cases} \mathbf{1}_\sharp, & w = \lambda, \\ s(w), & w \in \Sigma^+. \end{cases}$$

It is surjective, and $\mathsf{S}^\sharp \cong \Sigma^* / \ker \widehat{s}$. Here $\ker \widehat{s}$ denotes the kernel congruence

$$\ker \widehat{s} = \{(x, y) \in \Sigma^* \times \Sigma^* : \widehat{s}(x) = \widehat{s}(y)\}.$$

In particular,

$$\widehat{s}(x) = \widehat{s}(y) \quad \Longleftrightarrow \quad (x = y = \lambda) \text{ or } (x, y \in \Sigma^+ \text{ and } s(x) = s(y)).$$

Thus Definition 2.1 is equivalent to the all-fragment condition

$$\forall x, y \in \Sigma^*, \quad \big(\widehat{s}(x) = \widehat{s}(y) \wedge \mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset\big) \Longrightarrow \mathcal{D}_L(x) = \mathcal{D}_L(y).$$

Conversely, every finite-monoid typing $h : \Sigma^* \to M$ induces the finite-semigroup typing $s = h|_{\Sigma^+} : \Sigma^+ \to h(\Sigma^+)$. If substitutability relative to h compares nonempty *factors, as in Definition 2.1, it defines exactly the same class and nonempty-factor reconstruction. No equivalence is asserted for the different condition that also compares λ with nonempty factors of type $h(\lambda)$.*

Proof. Adjoining the identity extends multiplication and makes $\widehat{s}$ a monoid morphism. Every element of S is attained by a positive word, while λ attains $\mathbf{1}_\sharp$, so the map is surjective. The displayed kernel description follows because $\mathbf{1}_\sharp \notin \mathsf{S}$. Equal extended types compare either λ with itself or nonempty words of equal s-type, proving equivalence with Definition 2.1. For the converse, restricting h preserves all nonempty equalities, which alone define substitutability and the reconstruction rules. □

Throughout, subscripts of substitutability classes, reconstruction operators, and typed-thickness parameters denote semigroup morphisms on *nonempty* words. Write s_0 for the unique one-element semigroup typing, on whichever alphabet is in use. When a monoid morphism $h : \Sigma^* \to M$ is convenient for recognizing a regular language or evaluating empty contexts, its restriction $s = h|_{\Sigma^+}$ is the learning bias. The extension $\widehat{s}$ handles nullable yields (Lemma 2.2); algebraic conditions are tested on the positive image, not on its completion.

For $x \in \Sigma^*$ we call $\widehat{s}(x)$ its *yield type*; for nonempty x it equals $s(x)$. As above, $\mathcal{C}_s^{\mathrm{cf}}$ denotes the context-free $\sim_s$-substitutable languages, and we define $\mathcal{C}_s^{\mathrm{lin}} := \{L \in \mathcal{C}_s^{\mathrm{cf}} : L \text{ is generated by some linear CFG}\}$.

3 Fixed Recognizable Substitutability

For fixed s, put $\mathsf{RS}_s := \{L \subseteq \Sigma^* : L \text{ is } \sim_s\text{-substitutable}\}$. Then the fixed-$s$ slice is $\mathcal{C}_s^{\mathrm{cf}} = \mathsf{RS}_s \cap \mathrm{CFL}$, while $\mathcal{C}_s^{\mathrm{lin}}$ consists of its linearly generated members. For expressive comparisons only, let $\mathsf{RS} := \bigcup_s \mathsf{RS}_s$, where s ranges over finite-semigroup homomorphisms on Σ^+. We write $\mathsf{RS}_s(\Gamma)$ and $\mathsf{RS}(\Gamma)$ when the alphabet matters; all inclusions below are alphabetwise. The learning results always concern one fixed s.

Proposition 3.1. *A language $L \subseteq \Sigma^*$ is regular iff $L = \widehat{s}^{-1}(\mathsf{Acc})$ for some finite-semigroup typing $s : \Sigma^+ \to \mathsf{S}$ and some $\mathsf{Acc} \subseteq \mathsf{S}^\sharp$. Every language recognized in this way belongs to RS_s, so all regular languages belong to* RS.

Proof. For regular L, restrict its syntactic-monoid morphism [12, Chapter III, Proposition 10.2] to Σ^+, replace the codomain by its positive image S, and extend by a fresh identity. The positive types still determine membership for nonempty words; include $\mathbf{1}_\sharp$ among the accepting types exactly if $\lambda \in L$. Conversely, $\widehat{s}$ recognizes a regular language since $\mathsf{S}^\sharp$ is finite. For nonempty x, y with $s(x) = s(y)$, $\widehat{s}(uxv) = \widehat{s}(uyv)$ for all contexts $u, v \in \Sigma^*$, hence $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. □

By Proposition 3.1, RS and $\mathsf{RS} \cap \mathrm{CFL}$ contain every finite language and Σ^*; Gold's theorem makes both unions nonlearnable from positive data on nonempty alphabets. Yoshinaka [3, Section 4] notes the analogous issue for the union of fixed-window classes. We learn only fixed-s slices.

The fresh-identity completion reduces refinement and product typing to ordinary monoid-kernel operations. For finite-semigroup typings s, s', t of Σ^+, where each codomain is reduced to its positive image,

$$\ker \widehat{s'} \subseteq \ker \widehat{s} \iff \ker s' \subseteq \ker s, \qquad \ker \widehat{s \times t} = \ker \widehat{s} \cap \ker \widehat{t}.$$

Here $\widehat{s \times t}$ completes the *image* $(s \times t)(\Sigma^+)$ with a fresh identity. In fact its image monoid is isomorphic to $\mathrm{im}(\widehat{s} \times \widehat{t})$, a submonoid of $\mathsf{S}^\sharp \times \mathsf{T}^\sharp$. Consequently $\mathsf{RS}_s \subseteq \mathsf{RS}_{s'}$ under the first

inclusion, and $\mathsf{RS}_s \cup \mathsf{RS}_t \subseteq \mathsf{RS}_{s\times t}$. A refinement may increase the typed grammar size used in quantitative bounds.

For fixed k, ℓ, Yoshinaka [3, Proposition 1] showed that the class of all fixed-window substitutable languages$^{\nearrow}$ is closed under intersection of two members of the class and under inverse images of nonerasing homomorphisms. The same proposition shows that the class is not closed under intersection with arbitrary regular languages or under arbitrary inverse homomorphisms; the latter counterexample uses erasure. Yoshinaka also notes that the context-free subclass need not be closed under intersection. Proposition 3.2 instead records finite-typing closure facts that permit regular filtering and erasing inverse images by changing the typing.

Proposition 3.2 (closure under finite information)**.** *Let $s : \Sigma^+ \to \mathsf{S}$ and $t : \Sigma^+ \to \mathsf{T}$ be finite-semigroup typings.*

(i) *If $L \in \mathsf{RS}_s$ and $J \in \mathsf{RS}_t$, then $L \cap J \in \mathsf{RS}_{s\times t}$.*

(ii) *Let $\mathsf{Acc}_t \subseteq \mathsf{T}^\sharp$ be arbitrary and put $R_{\mathrm{reg}} := \widehat{t}^{-1}(\mathsf{Acc}_t)$. If $L \in \mathcal{C}^{\mathrm{cf}}_s$, then $L \cap R_{\mathrm{reg}} \in \mathcal{C}^{\mathrm{cf}}_{s\times t}$.*

(iii) *Let $\varphi : \Gamma^* \to \Sigma^*$ be any homomorphism, possibly erasing, and put $r = (\widehat{s} \circ \varphi)|_{\Gamma^+} : \Gamma^+ \to \mathsf{S}^\sharp$. If $L \in \mathsf{RS}_s$, then $\varphi^{-1}(L) \in \mathsf{RS}_r$; if moreover $L \in \mathcal{C}^{\mathrm{cf}}_s$, then $\varphi^{-1}(L) \in \mathcal{C}^{\mathrm{cf}}_r$.* [1]

Proof. For (i), nonempty factors of equal $(s \times t)$-type with a common context in $L \cap J$ have equal distributions in each language, and hence in their intersection. Item (ii) follows from (i) and closure of CFLs under regular intersection. For (iii), r is a semigroup morphism, since $\widehat{s} \circ \varphi$ is a monoid morphism. If two nonempty factors of equal r-type share a context in $\varphi^{-1}(L)$, their images are either both empty or both nonempty and of equal s-type. In the first case their inverse-image distributions coincide. In the second case, substitutability of L and the shared image context give the same conclusion after pulling back contexts. The context-free assertion uses closure of CFLs under inverse homomorphism. □

Typing adequacy. By Lemma 2.2, $L \in \mathsf{RS}_s$ exactly when $\ker \widehat{s}$ distinguishes every pair of factors with intersecting but unequal distributions. Thus typing adequacy is a finite-index congruence separation condition, preserved under refinement and products at a possible quantitative cost (Proposition 6.4). The stronger condition $\ker \widehat{s} \subseteq \equiv_L$ is sufficient but forces L to be regular.

3.1 Classical special cases

Clark–Eyraud's original Definition 5 includes all fragments [2]; we follow Clark [16] in excluding λ. This distinction matters even for $L = a^+$: (a, λ) belongs to both $\mathcal{D}_L(\lambda)$ and $\mathcal{D}_L(a)$, whereas (λ, λ) belongs to $\mathcal{D}_L(a)$ but not to $\mathcal{D}_L(\lambda)$. Thus comparing λ with nonempty factors *without type separation* would exclude a^+; $\widehat{s}$ makes those factors incomparable in type. Call L *Clark–Eyraud substitutable* when $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ for all $x, y \in \Sigma^+$. This is exactly the trivial-semigroup case of Definition 2.1, and hence also Yoshinaka's $(0, 0)$ case.

For $k, \ell \geq 0$, let $\mathrm{pref}_k(w)$ and $\mathrm{suf}_\ell(w)$ denote the prefix and suffix of lengths $\min\{k, |w|\}$ and $\min\{\ell, |w|\}$, respectively; in particular, $\mathrm{pref}_0(w) = \mathrm{suf}_0(w) = \lambda$. Renaming Yoshinaka's variables so that u_i, v_i denote outer contexts, L is *(k, ℓ)-substitutable* if, for all $p \in \Sigma^k$, $q \in \Sigma^\ell$, and $u_1, u_2, v_1, v_2, r_1, r_2 \in \Sigma^*$ with $pr_1q, pr_2q \neq \lambda$,

$$u_1pr_1qv_1 \in L, \quad u_1pr_2qv_1 \in L, \quad u_2pr_1qv_2 \in L \\ \implies \quad u_2pr_2qv_2 \in L.$$

Proposition 3.3 (finite-semigroup representation of fixed windows)**.** *For every $k, \ell \geq 0$ there is a surjective finite-semigroup morphism $s_{k,\ell} : \Sigma^+ \to W_{k,\ell}$ such that $L \subseteq \Sigma^*$ is (k, ℓ)-substitutable if and only if it is $\sim_{s_{k,\ell}}$-substitutable.*

[1] The fresh identity in $\mathsf{S}^\sharp$ replaces a separate erasure flag. For trivial s, $L = a^+$, $\varphi(a) = a$ and $\varphi(b) = \lambda$, the two nonempty inverse-image factors a, b share a context but have different distributions.

Proof. Put $m_0 := \max\{1, k+\ell\}$. For $w \in \Sigma^+$ define

$$s_{k,\ell}(w) := \begin{cases} (\text{short}, w), & 0 < |w| < m_0, \\ (\text{long}, \text{pref}_k(w), \text{suf}_\ell(w)), & |w| \geq m_0. \end{cases}$$

The displayed equivalence on Σ^+ is a two-sided congruence: equal short types represent the same word; equal long types retain their length-k prefix and length-ℓ suffix after any common left and right concatenation. Its finite quotient $W_{k,\ell}$ thus inherits an associative product, and $s_{k,\ell}$ is a surjective semigroup morphism.

For nonempty x, y, equal types mean either $x = y$ or $x = pr_1q$, $y = pr_2q$ for common $p \in \Sigma^k$, $q \in \Sigma^\ell$ and $r_1, r_2 \in \Sigma^*$. In the latter case a common context and an arbitrary context of one factor instantiate Yoshinaka's four-word condition; symmetry gives equal distributions. Conversely, the nonempty factors of any such premise have equal $s_{k,\ell}$-types, so relative substitutability supplies the fourth word. □

Yoshinaka [8, Example 4] already represented (k, ℓ) -substitutability by the sorted signature $\Sigma_{k,\ell}$. Proposition 3.3 makes the inducing finite-semigroup quotient explicit; no priority for the window-signature encoding is claimed.

Fixed-window counterexample criterion. If x, y have length at least $\max\{1, k+\ell\}$, the same length-k prefix and length-ℓ suffix, a common context, and unequal distributions, then L is not (k, ℓ)-substitutable: the first three conditions give equal $s_{k,\ell}$-types and a shared context, while the last contradicts $\sim_{s_{k,\ell}}$-substitutability.

Thus the fixed (k, ℓ) class is exactly $\mathsf{RS}_{s_{k,\ell}}$. Section 7 gives its polynomial thick-data bound. For $k = \ell = 0$, $s_{0,0}$ maps all nonempty words to one type, so it coincides with s_0 on Σ^+. For an alphabet Γ, set

$$\mathsf{KL}(\Gamma) := \bigcup_{k,\ell \geq 0} \{L \subseteq \Gamma^* : L \text{ is } (k, \ell)\text{-substitutable}\},$$

and abbreviate $\mathsf{KL} := \mathsf{KL}(\Sigma)$.

3.2 Main theorem

Theorem 3.4 (fixed-s learning theorem)**.** *Fix an efficiently computable finite-semigroup homomorphism $s : \Sigma^+ \twoheadrightarrow \mathsf{S}$. There are a set-driven reconstruction operator Rec_s and a conservative sequential learner Learn_s such that:*

(i) *Rec_s is sample-consistent and $\mathsf{Rec}_s(K)$ is constructible in time polynomial in $\|K\|$;*

(ii) *every $L \in \mathcal{C}_s^{\mathrm{cf}}$ has a finite characteristic sample for Rec_s;*

(iii) *Learn_s identifies every nonempty $L \in \mathcal{C}_s^{\mathrm{cf}}$ in Gold's sense, each update taking time polynomial in the data seen so far;*

(iv) *if the positive image semigroup $s(\Sigma^+)$ is a homomorphic image of a subsemigroup of the direct product of a finite locally trivial semigroup and a finite group, then for every nonempty target the sample in (ii) can be chosen polynomial in $|G_*|$ and $\tau_{G_*} + 1$ for any reduced CFG G_* representing that target; this includes every fixed-window and every finite-group typing;*

(v) *for arbitrary fixed s and $L \in \mathcal{C}_s^{\mathrm{lin}}$, the sample in (ii) can be chosen polynomial in $|G_*|$ for any linear CFG G_* representing L.*

The polynomials in (iv)–(v) may depend on the fixed s and Σ, but not on the chosen target grammar G_*. The quantitative mechanism behind items (iv)–(v), together with its limitation for arbitrary fixed typings, is developed in Sections 6–8.

This is consistent with Gold's theorem: on a nonempty alphabet choose distinct $x, y \in \Sigma^+$ with $s(x) = s(y)$, and choose $z \in \Sigma^*$ with $|z| > \max\{|x|, |y|\}$. The finite language $\{x, y, zx\}$ is not $\sim_s$-substitutable : x, y share the empty context but only x admits (z, λ).

Algebraic position of fixed windows. The locally trivial-semigroup identities underlying the following criterion and the corresponding prefix–suffix characterization of recognizable languages are classical [17, Chapter 2, Proposition 3.5 and Theorem 3.6]; see also the generalized definite language characterization [18, Proposition 5.18]. We state the positive-image/kernel formulation explicitly for arbitrary finite-semigroup typings and prove it below. Its learning-specific application is Corollary 7.5. These algebraic conditions must be tested on $\mathsf{S} = s(\Sigma^+)$, *not* on $\mathsf{S}^\sharp$. Indeed, for nonempty S, the new identity $\mathbf{1}_\sharp$ and any $t \in \mathsf{S}$ satisfy $\mathbf{1}_\sharp t \mathbf{1}_\sharp = t \neq \mathbf{1}_\sharp$; thus $\mathsf{S}^\sharp$ itself is not locally trivial.

Proposition 3.5 (locally trivial typings and fixed windows)**.** *Let $s : \Sigma^+ \twoheadrightarrow \mathsf{S}$ be a finite-semigroup typing on a nonempty alphabet and set $n := |\mathsf{S}| + 1$. The following are equivalent:*

(i) *S is locally trivial;*

(ii) *$\ker s_{k,\ell} \subseteq \ker s$ for some $k, \ell \geq 0$.*

In (ii), $k = \ell = n$ suffices. Every window semigroup $W_{k,\ell} = s_{k,\ell}(\Sigma^+)$ is locally trivial. Consequently,

$$\mathsf{KL}(\Sigma) = \bigcup_{\substack{s:\Sigma^+\to\mathsf{S} \\ \mathsf{S}\ \text{finite locally trivial}}} \mathsf{RS}_s(\Sigma),$$

also after intersecting both sides with CFL.

Proof. A short-word class in $W_{k,\ell}$ cannot be idempotent: squaring a positive word either changes its short representative or reaches the long part. If $e = s_{k,\ell}(x)$ is a long type, then xux retains the prefix and suffix of x for every $u \in \Sigma^+$. Thus $ete = e$ for all $t \in W_{k,\ell}$.

For every finite semigroup S, the classical identity $\mathsf{S}^n = \mathsf{S}E(\mathsf{S})\mathsf{S}$ holds for $n = |\mathsf{S}| + 1$ [17, Chapter 1, Proposition 1.12]. For locally trivial S, one also has $etf = ef$ for all idempotents e, f and $t \in \mathsf{S}$ [17, Chapter 2, Proposition 3.5]. For $p, q \in \Sigma^n$, write $s(p) = \alpha e \beta$, $s(q) = \gamma f \delta$ with e, f idempotent. For every $r \in \Sigma^*$, including λ, $\beta \widehat{s}(r) \gamma \in \mathsf{S}$, so $s(prq) = \alpha e (\beta \widehat{s}(r) \gamma) f \delta = \alpha e f \delta = s(pq)$. Hence length-$n$ windows determine the type of every word of length at least $2n$; shorter words are separated by $s_{n,n}$. Conversely, (ii) induces a surjective semigroup morphism $W_{k,\ell} \to \mathsf{S}$, preserving local triviality. The union identities follow by kernel refinement. □

4 Reconstruction Operator and Sequential Learner

4.1 Finite-sample reconstruction

For a finite sample $K \subseteq \Sigma^*$, let

$$\widehat{V}(K) := \{[\![x]\!] : x \in \Sigma^+,\ \exists u, v \in \Sigma^* \text{ with } uxv \in K\}.$$

Thus there is one hypothesis nonterminal for each distinct nonempty factor observed in the sample; the double brackets are part of the nonterminal name and do not denote a quotient class or literal terminal symbols. For observed nonempty factors x, y, write

$$x \approx_{K,s} y \quad \Longleftrightarrow \quad s(x) = s(y) \ \text{ and } \ \mathcal{D}_K(x) \cap \mathcal{D}_K(y) \neq \emptyset.$$

The grammar $\mathsf{Rec}_s(K)$ has nonterminal set $\widehat{V}(K) \cup \{\widehat{S}\}$, start symbol $\widehat{S}$, and the following *set* of productions whenever every displayed bracketed nonterminal belongs to $\widehat{V}(K)$:

$$\begin{aligned}
&\text{(B)} && [\![xy]\!] \to [\![x]\!][\![y]\!], && x, y \in \Sigma^+, \\
&\text{(U)} && [\![x]\!] \to [\![y]\!], && x \approx_{K,s} y, \\
&\text{(L)} && [\![a]\!] \to a, && a \in \Sigma, \\
&\text{(S)} && \widehat{S} \to [\![w]\!], && w \in K \cap \Sigma^+.
\end{aligned}$$

If $\lambda \in K$, Rule (S) also contains $\widehat{S} \to \lambda$.

For finite samples $K \subseteq K'$, every nonterminal and production of $\mathsf{Rec}_s(K)$ also occurs in $\mathsf{Rec}_s(K')$. Consequently, $L(\mathsf{Rec}_s(K)) \subseteq L(\mathsf{Rec}_s(K'))$.

The construction follows the substring-indexed grammar scheme of Yoshinaka [3, Section 4, Algorithm 1], also recalled by Clark [16, Section 4, Algorithm 1]. For the semigroup typing $s = s_{k,\ell}$, the nontrivial instances of Rule (U) coincide with Yoshinaka's unary rules: distinct observed nonempty factors are connected when they share a sample context and have equal $s_{k,\ell}$-type; equivalently, both factors have length at least $\max\{1, k+\ell\}$ and have the same length-k prefix and length-ℓ suffix. The constructions agree up to redundant identity rules, the representation of initial productions by a separate start symbol, and the treatment of λ. For general fixed s, the prefix–suffix test is replaced by equality of s-types.

Lemma 4.1 (sample consistency)**.** *For every finite sample K, $K \subseteq L(\mathsf{Rec}_s(K))$.*

Proof. If $w = a_1 \cdots a_m \in K \cap \Sigma^+$, Rule (S) enters $[\![w]\!]$. Repeated use of Rule (B) along any binary factorization of w reaches the one-letter factors, all of which are observed because they occur inside the same sample word. Rule (L) then emits $a_1 \cdots a_m$. If $\lambda \in K$, the start rule emits it directly. □

Theorem 4.2 (soundness)**.** *If $K \subseteq L$ and L is $\sim_s$-substitutable, then $L(\mathsf{Rec}_s(K)) \subseteq L$.*

Proof. Induct on the height of a derivation tree rooted at $[\![x]\!]$ to prove

$$[\![x]\!] \Rightarrow^*_{\mathsf{Rec}_s(K)} w \quad \Longrightarrow \quad x \equiv_L w \ \text{ and } \ s(w) = s(x). \tag{1}$$

Every factor nonterminal is indexed by a nonempty observed word, so the fragments compared by Rule (U) satisfy the nonempty requirement of Definition 2.1. Moreover, Rules (B)–(L) cannot derive λ from a factor nonterminal. Rule (L) is immediate. For Rule (U), suppose $[\![x]\!] \to [\![y]\!]$. By definition there is a sample context (u, v) with $uxv, uyv \in K \subseteq L$ and $s(x) = s(y)$. Fixed-s substitutability gives $x \equiv_L y$; the induction hypothesis gives $y \equiv_L w$ and $s(w) = s(y)$, proving (1).

For Rule (B), write the derived word as $w_1 w_2$ from $[\![xy]\!] \to [\![x]\!][\![y]\!]$. The induction hypotheses give $x \equiv_L w_1$, $y \equiv_L w_2$, $s(w_1) = s(x)$, and $s(w_2) = s(y)$. Because $\equiv_L$ is a monoid congruence, $xy \equiv_L w_1 w_2$, and the homomorphism law gives $s(w_1 w_2) = s(xy)$.

Finally, if a start rule $\widehat{S} \to [\![x]\!]$ is used, then $x \in K \subseteq L$, so $(\lambda, \lambda) \in \mathcal{D}_L(x)$. The invariant gives $\mathcal{D}_L(w) = \mathcal{D}_L(x)$ and hence $w \in L$. The direct λ rule is present only when $\lambda \in K \subseteq L$. □

4.2 The sequential learner

Recomputing Rec_s after every datum need not stabilize syntactically: for $L_c = \{a^n c b^n : n \geq 0\}$, longer examples introduce new factor nonterminals such as $[\![a^n c b^n]\!]$. We therefore use the conservative wrapper of Clark–Eyraud's Substitution Graph Learner (SGL, Algorithm 2) and Yoshinaka's (k,ℓ)-Substitutable Grammar Learner ((k,ℓ)-SGL, Algorithm 1) [2, 3]; see also Eyraud–Heinz–Yoshinaka [14, Appendix] for the distinction between set-driven reconstruction and incremental learning. Starting from the fixed empty grammar H_0, after reading w_n put

$$H_n := \begin{cases} H_{n-1}, & w_n \in L(H_{n-1}), \\ \mathsf{Rec}_s(K_n), & w_n \notin L(H_{n-1}), \end{cases} \qquad \mathsf{Learn}_s(\mathcal{T}[n]) := H_n.$$

Every rebuild uses the whole accumulated sample K_n. Although Rec_s is set-driven, Learn_s can stabilize on different grammars for different presentation orders. Indeed, let $L \in \mathcal{C}_s^{\mathrm{cf}}$ be infinite and let $\mathcal{T}$ be a text for L . Choose $w \in L$ longer than every sample word observed at the learner's last rebuild. The limiting grammar for $\mathcal{T}$ has no symbol $[\![w]\!]$, whereas the text obtained by prepending w rebuilds on a sample containing w and therefore retains $[\![w]\!]$ in its limiting grammar. Both hypotheses generate L; only eventual syntactic stabilization is asserted (Corollary 5.4). See also Clark [16, Section 4, Example 1].

5 Finite Witnesses, Exact Reconstruction, and Complexity

5.1 Target refinement and finite witnesses

For qualitative completeness, type an arbitrary target CFG over $\mathsf{S}^\sharp$. This refinement is used only in the completeness proof to show that the already-defined hypothesis grammar can simulate typed target derivations; it is not part of the learning algorithm. The hypothesis grammar $\mathsf{Rec}_s(K)$ remains unchanged, using nonempty observed factors and a direct start rule for λ.

Let $G = (V, \Sigma, P, S_0)$ be a CFG for a nonempty L. Write a rule as $\rho : A \to u_0 B_1 u_1 \cdots B_r u_r$ with $u_i \in \Sigma^*$, $B_i \in V$, and $r \geq 0$. For each $(\mu_1, \ldots, \mu_r) \in (\mathsf{S}^\sharp)^r$, set $\mu = \widehat{s}(u_0) \prod_{i=1}^{r} (\mu_i \widehat{s}(u_i))$ and include

$$A_\mu \to u_0 (B_1)_{\mu_1} u_1 \cdots (B_r)_{\mu_r} u_r.$$

This also covers terminal-only, empty, and unit rules. Add a fresh start symbol $\widetilde{S}$ with $\widetilde{S} \to (S_0)_\mu$ for each $\mu \in \mathsf{S}^\sharp$, and trim to the reduced grammar $\widetilde{G}$. This refinement is finite but can have exponential size; it is not part of the learning algorithm.

Every retained typed symbol is reachable and productive. Its yields have the indicated type; in particular, $A_{\mathbf{1}_\sharp}$ yields only λ, while A_μ with $\mu \neq \mathbf{1}_\sharp$ yields no empty word. For each retained non-start symbol X, choose the shortlex-minimal yield $\omega(X)$, possibly λ. Typing ensures that every yield of X, in particular $\omega(X)$, has the type assigned to X. Surrounding contexts carry no type annotations.

Proposition 5.1 (typed lifting and yield invariant)**.** *The typed refinement of any CFG G satisfies $L(\widetilde{G}) = L(G)$. For every retained A_μ, including typed copies of S_0,*

$$L_{\widetilde{G}}(A_\mu) = L_G(A) \cap \widehat{s}^{-1}(\{\mu\}).$$

In particular, $A_\mu \Rightarrow_{\widetilde{G}}^* w$ entails $\widehat{s}(w) = \mu$.

Proof. Erasing types gives a G-derivation; conversely, annotating each node by its terminal-yield type gives a typed derivation, including nullable children. Successful typed derivations survive trimming. For a retained A_μ, graft any annotated derivation of $w \in L_G(A)$ of type μ at a reachable occurrence of A_μ. This establishes the fiber and start-language equalities. □

For each retained non-start symbol X, choose a *canonical reaching context* $\chi(X) = (u_X, v_X)$ minimizing $|u_X| + |v_X|$, with shortlex tie breaks on u_X and then v_X. It exists because $\widetilde{G}$ was trimmed. For each typed rule $\rho : X \to u_0 Y_1 u_1 \cdots Y_r u_r$ $(X \neq \widetilde{S})$, let its *canonical right-hand-side yield* be $t_\rho = u_0 \omega(Y_1) u_1 \cdots \omega(Y_r) u_r$. Both t_ρ and $\omega(X)$ belong to $L_{\widetilde{G}}(X)$ and have the same type. For nonempty-type X, these yields are nonempty and occur in their shared canonical reaching context $\chi(X)$; the corresponding witnesses therefore license the unary Rule (U) from $[\![\omega(X)]\!]$ to $[\![t_\rho]\!]$. For empty-type X, both yields are λ and no factor nonterminal or unary rule is needed. Thus typing is used for this witness-based simulation, not claimed to be an algorithm-independent necessity. Define the finite set of *canonical positive witnesses*

$$\begin{aligned} \mathsf{Wit}(\widetilde{G}) := {} & \{u_X \omega(X) v_X : X \in \widetilde{V} \setminus \{\widetilde{S}\}\} \\ & \cup \{u_X t_\rho v_X : \rho = (X \to \alpha) \in \widetilde{P},\ X \neq \widetilde{S}\}. \end{aligned} \tag{2}$$

Every witness belongs to L. For an empty-type rule, $t_\rho = \omega(X) = \lambda$, so its witness is redundant. If $\lambda \in L$, the start copy $(S_0)_{\mathbf{1}_\sharp}$ has context (λ, λ) and yield λ; hence $\lambda \in \mathsf{Wit}(\widetilde{G})$.

5.2 Exact reconstruction and Gold convergence

Theorem 5.2 (completeness from the finite witnesses)**.** *Let G be any CFG generating a nonempty language L, and $\widetilde{G}$ its $\mathsf{S}^\sharp$-typed refinement. If $\mathsf{Wit}(\widetilde{G}) \subseteq K$, then $L \subseteq L(\mathsf{Rec}_s(K))$.*

Neither substitutability nor $K \subseteq L$ is required.

Proof. For each retained nonempty-type $X = A_\mu$, put $\widehat{X} := [\![\omega(X)]\!]$, an observed nonempty factor by the first witness family. Induct on the height of a typed derivation tree to show that $X \Rightarrow_{\widetilde{G}}^* w$ implies $\widehat{X} \Rightarrow_{\mathsf{Rec}_s(K)}^* w$.

Let the root rule be $\rho : X \to u_0 Y_1 u_1 \cdots Y_r u_r$, with child yields w_i. Both $\omega(X)$ and $t_\rho = u_0\omega(Y_1)u_1 \cdots \omega(Y_r)u_r$ are nonempty, have the same s-type, and occur in the shared context (u_X, v_X) in K. Thus Rule (U) gives $\widehat{X} \to [\![t_\rho]\!]$.

Delete the empty entries from $(u_0, \omega(Y_1), u_1, \ldots, \omega(Y_r), u_r)$, and denote the remaining entries in order by $z_1, \ldots, z_\nu$. Since X has nonempty type, $\nu \geq 1$ and $t_\rho = z_1 \cdots z_\nu$. Each entry and every concatenation of consecutive entries is an observed nonempty factor of $u_X t_\rho v_X \in K$. Repeated Rule (B) therefore gives $[\![t_\rho]\!] \Rightarrow_{\mathsf{Rec}_s(K)}^* [\![z_1]\!] \cdots [\![z_\nu]\!]$ (zero steps if $\nu = 1$). Terminal blocks are emitted by Rules (B) and (L). For a positive-type child Y_i, its corresponding entry is $z_j = \omega(Y_i)$, so $[\![z_j]\!] = \widehat{Y}_i \Rightarrow^* w_i$ by induction. Empty-type children have $w_i = \lambda$ and are omitted. Hence $\widehat{X} \Rightarrow^* u_0 w_1 u_1 \cdots w_r u_r = w$.

Every nonempty target word w has a typed start copy $(S_0)_{\widetilde{s}(w)}$ with canonical context (λ, λ). Its canonical yield belongs to K, so start Rule (S) and the induction give a derivation of w. If $\lambda \in L$, its start-copy witness puts $\lambda \in K$, and Rule (S) emits it directly. □

Theorem 5.3 (exact finite-sample reconstruction)**.** *Let $L \in \mathcal{C}_s^{\mathrm{cf}}$ be nonempty and let G be any CFG for L, with $\mathsf{S}^\sharp$-typed refinement $\widetilde{G}$. If $\mathsf{Wit}(\widetilde{G}) \subseteq K \subseteq L$, then $L(\mathsf{Rec}_s(K)) = L$.*

Proof. Apply Theorem 5.2 to G and Theorem 4.2 to $K \subseteq L$. □

Thus $\mathsf{Wit}(\widetilde{G})$ is characteristic for every nonempty target, including $\{\lambda\}$. For $L = \emptyset$, the empty sample gives $L(\mathsf{Rec}_s(\emptyset)) = \emptyset$. This proves Theorem 3.4(ii).

Corollary 5.4 (Gold identification)**.** *For a fixed surjective semigroup homomorphism $s : \Sigma^+ \to \mathsf{S}$ to a finite semigroup, the learner Learn_s identifies every nonempty language in $\mathcal{C}_s^{\mathrm{cf}}$ from positive data. More precisely, for any CFG G representing the target and its general typed refinement $\widetilde{G}$, once $\mathsf{Wit}(\widetilde{G}) \subseteq K_{n_0}$, among stages $n \geq n_0$ at most one hypothesis change can occur.*

Proof. Let $\mathcal{T}$ be a text for a nonempty L , and choose n_0 with $\mathsf{Wit}(\widetilde{G}) \subseteq K_{n_0}$. Since $w_1 \notin L(H_0) = \emptyset$, the learner rebuilds at stage 1, so for $n \geq 1$ the hypothesis H_n is $\mathsf{Rec}_s(K_m)$ for the most recent rebuild stage $m \leq n$. Sample consistency of that rebuild and the rebuild rule for the later data give $K_n \subseteq L(H_n)$, and soundness gives $L(H_n) \subseteq L$, as $K_m \subseteq L$. If a rebuild occurs at some $n \geq n_0$, Theorem 5.3 gives $L(H_n) = L$, and no later datum triggers a change. Otherwise $H_n = H_{n_0}$ for all $n \geq n_0$, so every word of $\mathcal{T}$ lies in some $K_n \subseteq L(H_n) = L(H_{n_0})$ with $n \geq n_0$; since every word of L occurs in $\mathcal{T}$, $L(H_{n_0}) = L$. In both cases at most one change occurs after stage n_0. □

5.3 Complexity of the reconstruction step

Theorem 5.5 (polynomial-time reconstruction)**.** *For fixed s, $\mathsf{Rec}_s(K)$ is constructible in time polynomial in $\|K\|$.*

Proof. Put $n_K := \|K\|$ and let $\mathcal{O}_K := \{(u, x, v) \in \Sigma^* \times \Sigma^+ \times \Sigma^* : uxv \in K\}$ be the set of observed nonempty factor occurrences. Since $|\mathcal{O}_K| = O(n_K^2)$, there are at most $O(n_K^2)$ factor nonterminals. Extending factors one symbol at a time from each starting position of each sample word computes and caches all their s-values using $O(n_K^2)$ multiplications in the fixed semigroup.

Rule (B) has at most $O(n_K^3)$ candidates, obtained by choosing an observed factor occurrence and a split point. For Rule (U), for each $(u, x, v) \in \mathcal{O}_K$ and each $w \in K$, check whether there is a nonempty word y with $w = uyv$. Such a y, if it exists, is unique; if $s(x) = s(y)$, emit the rule $[\![x]\!] \to [\![y]\!]$. This enumerates all Rule (U) candidates, at most $|\mathcal{O}_K||K| = O(n_K^3)$ of them.

Lexical and start rules are lower-order. Checking, encoding and inserting a candidate take $O(n_K)$ time, and a trie of the explicit rule encodings removes duplicates, so the output is a rule set independent of the enumeration order. Thus $\mathsf{Rec}_s(K)$ is constructible in $O(n_K^4)$ time. □

Each update of Learn_s is also polynomial in the data seen so far, since CFG membership is polynomial and hypotheses and rebuilds are polynomial by Theorem 5.5 . Together with sample consistency (Section 4), Theorem 5.3 and Corollary 5.4, this proves Theorem 3.4(i)–(iii); items (iv)–(v) are proved in Sections 7–8. Grammar size alone cannot bound witness lengths: $S_0 \to D_n$, $D_0 \to a$, $D_i \to D_{i-1}D_{i-1}$ has size $O(n)$ but generates $\{a^{2^n}\}$, substitutable for every s [2, Example 2], [14, Example 2.3].

6 Typed Thickness for General Fixed Typings

Definition 6.1 (binary–unit–nullable normal form)**.** *A CFG is in* binary–unit–nullable normal form *(BUNF) if every production is* $A \to BC$, $A \to B$, $A \to a$, *or* $A \to \lambda$.

Terminal isolation and binarization give a polynomial-size equivalent BUNF grammar; Proposition 7.3 also controls ordinary thickness. Here BUNF bounds typed-rule counts and derivation-tree branching, without restricting Section 5's learning results. Let $G = (V, \Sigma, P, S_0)$ be a reduced BUNF grammar, $\widetilde{G}$ its refinement, and put $N := |V|$ and $N_t := |\widetilde{V} \setminus \{\widetilde{S}\}|$. Define its *typed thickness* by

$$\tau_s^{\mathrm{typ}}(G) := \max\Big(\{\, |\omega(X)| : X \in \widetilde{V} \setminus \{\widetilde{S}\} \,\} \cup \{1\}\Big).$$

It is a presentation-dependent parameter, not an additional assumption on the learner. The typed thickness includes copies of the original start symbol, even if it occurs on a right-hand side or derives λ. In particular, every nonempty $L = L(G)$ satisfies

$$\tau_s^{\mathrm{typ}}(G) \geq \max_{\mu \in \widehat{s}(L)} \min\{\, |w| : w \in L,\ \widehat{s}(w) = \mu \,\}. \tag{♠}$$

Indeed, for each $\mu \in \widehat{s}(L)$, the typed start copy $(S_0)_\mu$ survives trimming, and Proposition 5.1 identifies its shortest yield length with the corresponding minimum on the right.

Lemma 6.2 (typed-thickness witness bound)**.** *Let G and $\widetilde{G}$ be as above, and put $\tau := \tau_s^{\mathrm{typ}}(G)$. Every non-start typed symbol X satisfies $|u_X| + |v_X| \leq (N_t - 1)\tau$, and each $w \in \mathsf{Wit}(\widetilde{G})$ satisfies $|w| \leq (N_t + 1)\tau$. Let P_{ter}, P_{unit} and P_{bin} be the terminal, unit and binary productions of G. Then*

$$\|\mathsf{Wit}(\widetilde{G})\| \leq \big(N|\mathsf{S}^\sharp| + |P_{\mathrm{ter}}| + |P_{\mathrm{unit}}||\mathsf{S}^\sharp| + |P_{\mathrm{bin}}||\mathsf{S}^\sharp|^2\big)\big((N|\mathsf{S}^\sharp| + 1)\tau + 1\big).$$

For fixed s, this is polynomial in $|G|$ and $\tau_s^{\mathrm{typ}}(G)$.

Proof. In the dependency graph of retained typed symbols include unit and binary dependencies. Each symbol is reachable from a typed copy of S_0; a shortest path has at most $N_t - 1$ edges. Expand off-path siblings of binary rules by canonical yields, each of length at most τ. Unit steps contribute no sibling. Hence the canonical reaching context has length at most $(N_t - 1)\tau$. Each BUNF rule adds at most two canonical yields or one terminal. Empty-type rule witnesses duplicate canonical-yield witnesses and need no extra count. There are at most $N|\mathsf{S}^\sharp|$ typed symbols, $|P_{\mathrm{ter}}|$ terminal typed rules, $|P_{\mathrm{unit}}||\mathsf{S}^\sharp|$ unit typed rules, and $|P_{\mathrm{bin}}||\mathsf{S}^\sharp|^2$ binary typed rules. Summing $|w| + 1$ over the witness set proves the claim. □

Corollary 6.3 (characteristic data via BUNF normalization). *Fix s. Let G_* be any reduced CFG for a nonempty $L(G_*) \in \mathcal{C}_s^{\mathrm{cf}}$, and let G be an equivalent reduced BUNF normalization of size polynomial in $|G_*|$. (If G_* is already BUNF, take $G = G_*$.) Then $\mathsf{Wit}(\widetilde{G})$ is a characteristic sample for Rec_s whose encoded size is polynomial in $|G_*|$ and $\tau_s^{\mathrm{typ}}(G)$.*

Proof. Applying Theorem 5.3 to G shows that $\mathsf{Wit}(\widetilde{G})$ is characteristic for $L(G) = L(G_*)$. Apply Lemma 6.2 to its chosen BUNF normalization G, using that $|G|$ is polynomial in $|G_*|$. □

The typed-thickness parameter in Corollary 6.3 belongs to the chosen BUNF normalization, not directly to G_*. For arbitrary fixed s, this does not imply a polynomial bound in the original grammar size and ordinary thickness; indeed, ordinary thickness cannot in general replace typed thickness even for the reconstruction operator.

A bracketed doubling family. Let $\Gamma_{\mathrm{br}} = \{\mathsf{a}, \lfloor, \rfloor, \lceil, \rceil\}$. Let $U_1 = \{1, \zeta\}$ be the two-element semilattice with identity 1 and absorbing element ζ: $1\zeta = \zeta 1 = \zeta^2 = \zeta$. Define the surjective semigroup morphism $s_{\mathrm{U}} : \Gamma_{\mathrm{br}}^+ \to U_1$ by sending $\mathsf{a}, \lfloor, \rfloor$ to 1 and $\lceil, \rceil$ to ζ. Thus $s_{\mathrm{U}}(w) = 1$ exactly for positive words without ceiling brackets, whereas $\widehat{s}_{\mathrm{U}}(\lambda) = \mathbf{1}_\sharp \neq 1$.

For $n \geq 1$, define the reduced CFG G_n by

$$S_0 \to E_n, \qquad E_0 \to \mathsf{a}, \qquad E_i \to \lfloor E_{i-1}E_{i-1} \rfloor \mid \lceil E_{i-1} \rceil \quad (1 \leq i \leq n),$$

and put $L_n = L(G_n)$. Then $|G_n| = O(n)$ and $\tau_{G_n} = 2n + 1$. Write $w_0^* = \mathsf{a}$ and $w_i^* = \lfloor w_{i-1}^* w_{i-1}^* \rfloor$. The only word of L_n without ceiling brackets is w_n^*, of length $|w_n^*| = 3 \cdot 2^n - 2$: without unary ceiling-bracket nodes the derivation is the unique complete binary tree of height n.

Proposition 6.4 (refinement can force exponential characteristic data). *For every $n \geq 1$ the following hold.*

(i) *L_n is Clark–Eyraud substitutable, and hence $L_n \in \mathcal{C}_{s_0}^{\mathrm{cf}} \cap \mathcal{C}_{s_{\mathrm{U}}}^{\mathrm{cf}}$.*

(ii) *L_n has a characteristic sample for Rec_{s_0} of encoded size polynomial in n.*

(iii) *Every finite $K \subseteq L_n$ such that $w_n^* \in L(\mathsf{Rec}_{s_{\mathrm{U}}}(K))$ contains w_n^*. Consequently, every characteristic sample $\mathcal{F}$ of L_n for $\mathsf{Rec}_{s_{\mathrm{U}}}$ satisfies $\|\mathcal{F}\| \geq 3 \cdot 2^n - 1$.*

(iv) *Put $L_n^- := L_n \setminus \{w_n^*\}$. Then $L_n^- \in \mathcal{C}_{s_{\mathrm{U}}}^{\mathrm{cf}}$. For any set-driven reconstruction operator Op with characteristic samples for both L_n and L_n^-, every characteristic sample $\mathcal{F}$ of L_n for Op contains w_n^* and hence satisfies $\|\mathcal{F}\| \geq 3 \cdot 2^n - 1$. In particular, this holds for every set-driven operator having characteristic samples for the entire fixed-s_{U} class.*

In particular, for the fixed typing s_{U} no bound on characteristic-data size for $\mathsf{Rec}_{s_{\mathrm{U}}}$ polynomial in source grammar size and ordinary thickness holds over all reduced CFG presentations of targets in $\mathcal{C}_{s_{\mathrm{U}}}^{\mathrm{cf}}$.

Proof. The substitutability assertion (i) is proved in Appendix C. For (ii), an explicit sample suffices. Set $u_0 = \mathsf{a}$, $u_i = \lceil u_{i-1} \rceil$, and

$$\mathcal{F}_n = \{u_n\} \cup \{\lceil^{n-i} \lfloor u_{i-1}u_{i-1} \rfloor \rceil^{n-i} : 1 \leq i \leq n\}.$$

For each i, the two sample words provide a common context for u_i and $\lfloor u_{i-1}u_{i-1} \rfloor$; Rules (U) and (B) therefore recover both E_i alternatives, so $L_n \subseteq L(\mathsf{Rec}_{s_0}(\mathcal{F}_n))$. For any $\mathcal{F}_n \subseteq K \subseteq L_n$, monotonicity applied to $\mathcal{F}_n \subseteq K$ gives $L_n \subseteq L(\mathsf{Rec}_{s_0}(K))$, while soundness (Theorem 4.2) applied to $K \subseteq L_n$ gives $L(\mathsf{Rec}_{s_0}(K)) \subseteq L_n$. Thus $\mathcal{F}_n$ is characteristic, with $|\mathcal{F}_n| = n + 1$ and $\|\mathcal{F}_n\| = 3n^2 + 4n + 2$.

For (iii)–(iv), the deleted-word target is the type-fiber filter $L_n^- = L_n \cap \widehat{s}_{\mathrm{U}}^{-1}(\{\zeta\})$. It belongs to $\mathcal{C}_{s_{\mathrm{U}}}^{\mathrm{cf}}$: equal s_{U}-types of nonempty factors remain equal $\widehat{s}_{\mathrm{U}}$-types inside any common outer context. Thus intersection with a union of $\widehat{s}_{\mathrm{U}}$-fibers preserves relative substitutability, and the intersection is context-free by regular filtering.

For (iii), if $K \subseteq L_n$ omits w_n^*, then $K \subseteq L_n^-$. Soundness (Theorem 4.2) gives $L(\mathsf{Rec}_{s_U}(K)) \subseteq L_n^-$, so w_n^* cannot be generated. Hence every characteristic sample $\mathcal{F}$ contains w_n^* and satisfies $\|\mathcal{F}\| \geq |w_n^*| + 1 = 3 \cdot 2^n - 1$. Together with $|G_n|, \tau_{G_n} = O(n)$, this proves the final assertion.

For (iv), let $\mathcal{F}$ be characteristic for L_n under Op and suppose $w_n^* \notin \mathcal{F}$. Take a characteristic sample $\mathcal{F}'$ for L_n^-. Then $\mathcal{F} \cup \mathcal{F}' \subseteq L_n^- \subsetneq L_n$, so the same output $\mathsf{Op}(\mathcal{F} \cup \mathcal{F}')$ would have to generate both languages, a contradiction.

For $n \geq 2$, $L_n^- \notin \mathcal{C}_{s_0}^{\mathrm{cf}}$: u_{n-1} and w_{n-1}^* share the context $(\lfloor, u_{n-1} \rfloor)$ in L_n^-, but only the former admits $(\lfloor w_{n-1}^*, \rfloor)$. Thus (iv) does not conflict with the s_0 upper bound in (ii). □

Remark 6.5 (finite targets and linear grammar size). Although each L_n is finite and hence linear as a language, the displayed $O(n)$-size grammar G_n is not linear. For a finite language generated by a linear CFG, a productive derivation with a repeated nonterminal on its unique spine cannot produce a nonempty terminal string around the cycle: iterating such a cycle would generate infinitely many words. Cycles producing only the empty word can be omitted. Consequently a derivation can be chosen without repeated nonterminals, and its terminal word length is bounded by the total number of terminal occurrences in the grammar's production listing. Since $|w_n^*| = 3 \cdot 2^n - 2$, every linear CFG generating L_n has exponentially many terminal occurrences in its productions. This is consistent with Theorem 8.3.

The contradiction in part (iv) follows the classical distinguishing-pair principle using a word in the symmetric difference of two targets; compare [4, Theorem 15.1.4]. The fixed-s distinguishing pair L_n, L_n^- and its exponential witness length are specific to the present construction.

Corollary 6.6 (presentation-independent typed-thickness gap)**.** *Every reduced BUNF grammar $G = (V, \Gamma_{\mathrm{br}}, P, S_0)$ for L_n satisfies*

$$\boxed{\tau_{s_U}^{\mathrm{typ}}(G) \geq 3 \cdot 2^n - 2.}$$

This holds regardless of grammar size. Conversely, L_n has a reduced BUNF presentation with both size and ordinary thickness polynomial in n.

Proof. The positive fiber of type 1 is $L_n \cap \widehat{s}_U^{-1}(\{1\}) = \{w_n^*\}$, with $1 \neq \mathbf{1}_\sharp$. The bound (♠) therefore gives $\tau_{s_U}^{\mathrm{typ}}(G) \geq |w_n^*| = 3 \cdot 2^n - 2$ for every such G. For the ordinary-thickness upper construction, apply Proposition 7.3 to G_n. □

7 Polynomial Thick Data Beyond Fixed Windows

Yoshinaka [3, Theorem 1] already established polynomial thick-data learnability for fixed (k, ℓ)-substitutable context-free languages using (k, ℓ)-Greibach normal form and characteristic-set bounds [3, Lemmas 2 and 7]. Here we instead bound the shortest yields of the typed refinement used by Rec_s. One argument covers both fixed windows and typings that additionally track finite-group values. The corresponding batch bounds establish identification in polynomial time and thick data (IPTtD), in the set-driven sense of [3] and [14, Definition 2.12]. They do not assert the stronger incremental stabilization requirement of [14, Definition 2.5]. We claim no improvement in Yoshinaka's polynomial degree. For general fixed s, ordinary thickness need not give a polynomial characteristic-data bound for Rec_s (Proposition 6.4).

Fix $k, \ell \geq 0$ and a morphism $g : \Sigma^* \to \mathsf{H}$ into a finite group. In products with $s_{k,\ell}$, the group component is restricted to Σ^+.

Lemma 7.1 (window–group typed-yield bound)**.** *Let $G = (V, \Sigma, P, S_0)$ be a reduced BUNF grammar, $N := |V| \geq 1$, $m := k + \ell$ and $q := |\mathsf{H}|$. If $\ker(s_{k,\ell} \times (g|_{\Sigma^+})) \subseteq \ker s$, then*

$$\tau_s^{\mathrm{typ}}(G) \leq m + 1 + [1 + 2(m+q)(m+2)Nq](\tau_G + 1).$$

This includes nullable and unit productions, including productions whose right-hand sides contain S_0.

Proof. See Appendix A, where a nonemptiness anchor and derivation-size tie break handle the additional BUNF cases. □

Theorem 7.2 (polynomial thick data under window–group refinement)**.** *Fix s, Σ and suppose $\ker\big(s_{k,\ell} \times (g|_{\Sigma^+})\big) \subseteq \ker s$ for some fixed k, ℓ and finite-group morphism g. Every nonempty $L \in \mathcal{C}_s^{\mathrm{cf}}$ represented by a reduced BUNF grammar G has a characteristic sample $\mathsf{Wit}(\widetilde{G})$ for Rec_s of encoded size polynomial in $|G|$ and $\tau_G + 1$.*

Proof. Combine Lemma 7.1 and Corollary 6.3. □

Theorem 7.2 includes fixed windows when H is trivial. For arbitrary CFG presentations we need only terminal isolation and binarization.

Proposition 7.3 (polynomial thickness-preserving BUNF normalization)**.** *There are fixed polynomials q_{size} and q_{thick} such that every reduced CFG G_* generating a nonempty language can be converted in polynomial time into an equivalent reduced BUNF grammar G with $|G| \le q_{\mathrm{size}}(|G_*|)$ and $\tau_G \le q_{\mathrm{thick}}(|G_*|, \tau_{G_*} + 1)$. The construction retains nullable and unit productions.*

See Appendix A.2.

Recall that a semigroup S *divides* a semigroup Q if S is a homomorphic image of a subsemigroup of Q.

Lemma 7.4 (locally trivial–group divisors and window refinements)**.** *Let $s : \Sigma^+ \twoheadrightarrow \mathsf{S}$ be a finite-semigroup typing on a nonempty alphabet. The following are equivalent:*

(i) *S divides a product $\mathsf{T} \times \mathsf{H}$, where T is a finite locally trivial semigroup and H is a finite group;*

(ii) *for some $k, \ell \ge 0$ and a monoid morphism $g : \Sigma^* \to \mathsf{H}$ into a finite group H,*

$$\ker\big(s_{k,\ell} \times (g|_{\Sigma^+})\big) \subseteq \ker s.$$

Proof. For (i), choose a subsemigroup $\mathsf{U} \subseteq \mathsf{T} \times \mathsf{H}$ and a surjective morphism $\pi : \mathsf{U} \to \mathsf{S}$. Lift the letter images $s(a)$ to U, obtaining $\varphi : \Sigma^+ \to \mathsf{U}$ with $\pi\varphi = s$. The factor projections are $\sigma : \Sigma^+ \to \mathsf{T}$ and $g_+ : \Sigma^+ \to \mathsf{H}$. The latter extends to $g : \Sigma^* \to \mathsf{H}$ with $g(\lambda) = 1_{\mathsf{H}}$. Since $\sigma(\Sigma^+)$ is locally trivial, Proposition 3.5 supplies k, ℓ with $\ker s_{k,\ell} \subseteq \ker \sigma$. Together with g_+ this implies (ii).

Conversely (ii) induces a surjective semigroup morphism

$$(s_{k,\ell} \times g)(\Sigma^+) \longrightarrow \mathsf{S}, \qquad (s_{k,\ell}(w), g(w)) \longmapsto s(w).$$

The domain is a subsemigroup of $W_{k,\ell} \times \mathsf{H}$, with $W_{k,\ell}$ locally trivial by Proposition 3.5; hence (i). □

Corollary 7.5 (ordinary-thickness bounds for locally trivial–group typings)**.** *Fix $s : \Sigma^+ \twoheadrightarrow \mathsf{S}$. Suppose S is a homomorphic image of a subsemigroup of the direct product of a finite locally trivial semigroup and a finite group. Then every nonempty $L \in \mathcal{C}_s^{\mathrm{cf}}$ represented by a reduced CFG G_* has characteristic data for Rec_s of encoded size polynomial in $|G_*|$ and $\tau_{G_*} + 1$. This includes all locally trivial positive-image typings and all finite-group typings.*

Proof. By Lemma 7.4, Theorem 7.2 applies to the BUNF grammar given by Proposition 7.3; compose the polynomials. □

Let **LI** and **Grp** be the pseudovarieties of finite locally trivial semigroups and of finite groups. As both are closed under finite direct products, their join $\mathbf{LI} \vee \mathbf{Grp}$ consists exactly of the divisors (homomorphic images of subsemigroups) of products $\mathsf{T} \times \mathsf{H}$ with $\mathsf{T} \in \mathbf{LI}$ and $\mathsf{H} \in \mathbf{Grp}$; such joins were studied by Costa [19, 20]. The hypothesis of Corollary 7.5 thus reads $s(\Sigma^+) \in \mathbf{LI} \vee \mathbf{Grp}$.

Lemma 7.6 (the two-element semilattice lies outside the window–group join)**.** $U_1 \notin \mathbf{LI} \vee \mathbf{Grp}$.

Proof. Suppose there were a subsemigroup $\mathsf{V} \subseteq \mathsf{T} \times \mathsf{H}$ and a surjective morphism $\pi : \mathsf{V} \to U_1$, with $\mathsf{T} \in \mathbf{LI}$ and H a finite group. Choose idempotents $e \in \pi^{-1}(1)$ and $f \in \pi^{-1}(\zeta)$ (Section 2.2). Every idempotent of $\mathsf{T} \times \mathsf{H}$ has group component 1_{H}, so the T-components of e, f are idempotents. Local triviality of T gives $efe = e$. Applying π would yield $1 \cdot \zeta \cdot 1 = 1$, contradicting $\zeta \neq 1$. □

Remark 7.7. The positive image of s_{U} is U_1, which lies outside $\mathbf{LI} \vee \mathbf{Grp}$ by Lemma 7.6. Proposition 6.4 shows that the ordinary-thickness characteristic-data bound in Corollary 7.5 can fail for Rec_s when its algebraic hypothesis is removed. This does not establish that the hypothesis is necessary for every fixed typing. Proposition 6.4(iv) does give a reconstruction-operator-independent lower bound on characteristic-sample size for the distinguishing target pair, but not a lower bound on the running time or convergence stage of arbitrary Gold learners.

These polynomial bounds may depend on the fixed typing and alphabet; no uniform bound for variable windows or groups is claimed.

8 The Linear Subclass

Linear languages are exactly the languages accepted by one-turn pushdown automata [21, 22]. The normalization below gives the grammatical counterpart: every successful derivation has one continuing nonterminal path, which we call its *spine*.

Proposition 8.1 (linear-spine BUNF normalization)**.** *There is a fixed polynomial q_{lin} such that every linear CFG G generating a nonempty language can be converted in polynomial time into an equivalent reduced BUNF grammar $G_0 = (V_0, \Sigma, P_0, S_0)$ with $|G_0| \le q_{\mathrm{lin}}(|G|)$ in which every binary production has exactly one fresh terminal-wrapper child W_a with $W_a \to a$ and exactly one other nonterminal child.*

The construction is given in Appendix B. Let $\widetilde{G}_0$ be its typed refinement and $N_t := |\widetilde{V}_0 \setminus \{\widetilde{S}\}| \le |V_0||\mathsf{S}^\sharp|$.

Lemma 8.2 (short canonical yields)**.** *Every typed non-start symbol X of $\widetilde{G}_0$ satisfies $|\omega(X)| \le N_t$. Hence $\tau_s^{\mathrm{typ}}(G_0) \le N_t$.*

Proof. The *continuing spine* follows the unique non-wrapper child of each binary rule. The wrapper child generates exactly one terminal and never continues the spine. Unit rules add no terminals; lexical and nullable rules end the spine. Choose a derivation of $\omega(X)$ with the fewest nonterminal occurrences. A repeated *typed* spine symbol permits grafting its lower subderivation at the upper occurrence, preserving a valid typed derivation. A segment containing a binary rule would strictly shorten $\omega(X)$; a unit-only segment would decrease the number of nodes. The former contradicts the minimal length of $\omega(X)$; the latter contradicts the choice of a derivation with the fewest nonterminal occurrences. At most N_t spine symbols occur, and each contributes at most one terminal, including the final lexical rule. Therefore $|\omega(X)| \le N_t$. Wrapper symbols also satisfy this estimate. □

Theorem 8.3 (polynomial characteristic data for linear targets)**.** *For fixed s, every $L \in \mathcal{C}_s^{\mathrm{lin}}$ represented by a linear CFG G has a characteristic sample for Rec_s whose $\|\cdot\|$-size is polynomial in $|G|$.*

Proof. For $L = \emptyset$ and $L = \{\lambda\}$ use the samples from Section 5. Otherwise take G_0 from Proposition 8.1. Lemma 8.2 gives $\tau_s^{\mathrm{typ}}(G_0) \le N_t \le |V_0|\,|\mathsf{S}^\sharp|$, and Corollary 6.3 yields a characteristic sample polynomial in $|G_0| \le q_{\mathrm{lin}}(|G|)$. More explicitly, the bound in Lemma 6.2 gives $\|\mathsf{Wit}(\widetilde{G}_0)\| = O(|G_0|^3 |\mathsf{S}^\sharp|^4)$; the polynomial dependence on the original $|G|$ also uses the normalization bound q_{lin}. □

Together with Theorem 5.5, this gives polynomial-time batch reconstruction and polynomial characteristic data relative to a chosen linear CFG for targets in $\mathcal{C}_s^{\mathrm{lin}}$. This does not contradict de la Higuera's negative result for the unrestricted class of linear languages [15, Theorem 2]: our theorem concerns only the fixed-s substitutable subclass. Moreover, no incremental polynomial-data guarantee for Learn_s is asserted.

A nonregular linear fixed-s example. The preceding theorem is not confined to regular targets, nor to the Clark–Eyraud or fixed-window classes. Define

$$L_{\pm,e} := \{a^{2m} c b^{2m} : m \ge 0\} \cup \{a^{2m+1} d b^{2m+1} : m \ge 0\} \cup \{a^n e b^n : n \ge 0\}.$$

It is generated by the linear grammar

$$S \to U_{\mathrm{ev}} \mid U_e, \quad U_{\mathrm{ev}} \to c \mid a U_{\mathrm{odd}} b, \quad U_{\mathrm{odd}} \to d \mid a U_{\mathrm{ev}} b, \quad U_e \to e \mid a U_e b.$$

The subscripts ev and odd track the parity of enclosing a,b-pairs from the start, not the length parity of the corresponding nonterminal's own yield.

Call c, d, e the *center symbols*.

Proposition 8.4. *The language $L_{\pm,e}$ is linear, nonregular, and belongs to $\mathcal{C}_s^{\mathrm{lin}}$ for some homomorphism s into a finite semigroup. It is neither Clark–Eyraud substitutable nor (k,ℓ)-substitutable for any fixed $k, \ell \ge 0$.*

Proof. Linearity follows from the displayed grammar. To establish finite-typed substitutability, put

$$L_{\mathrm{all}} := \{a^n \xi b^n : n \ge 0,\ \xi \in \{c,d,e\}\}, \qquad R_{\mathrm{reg}} := (aa)^* c (bb)^* \cup a(aa)^* d\, b(bb)^* \cup a^* e b^*.$$

The linear language L_{all} is Clark–Eyraud substitutable under the nonempty-factor convention: a shared context uniquely determines a center-free factor, while the distribution of a factor $a^i \xi b^j$ containing one center depends only on $i - j$. Thus $L_{\mathrm{all}} \in \mathcal{C}_{s_0}^{\mathrm{lin}}$ for the trivial typing s_0. Since R_{reg} is regular and $L_{\pm,e} = L_{\mathrm{all}} \cap R_{\mathrm{reg}}$, let $g : \Sigma^* \to M$ be a finite-monoid morphism recognizing R_{reg}. Put $\mathsf{T} = g(\Sigma^+)$ and $t = g|_{\Sigma^+} : \Sigma^+ \twoheadrightarrow \mathsf{T}$. Because $\lambda \notin R_{\mathrm{reg}}$, a suitable $\mathsf{Acc}_t \subseteq \mathsf{T} \subseteq \mathsf{T}^\sharp$ satisfies $R_{\mathrm{reg}} = \widetilde{t}^{-1}(\mathsf{Acc}_t)$. Proposition 3.2(ii), together with closure of linear languages under regular intersection, then gives $L_{\pm,e} \in \mathcal{C}_{s_0 \times t}^{\mathrm{lin}}$.

Moreover, $L_{\pm,e} \cap a^* e b^* = \{a^n e b^n : n \ge 0\}$ proves nonregularity. The factors c, e share the empty context, but $(a,b) \in \mathcal{D}_{L_{\pm,e}}(e) \setminus \mathcal{D}_{L_{\pm,e}}(c)$; hence $L_{\pm,e}$ is not Clark–Eyraud substitutable. Finally, fix $k, \ell \ge 0$ and put $m_0 := \max\{1, k+\ell\}$, $w_e := a^k e b^\ell$, and $w_c := a^k c b^\ell$. The factors w_e, w_c have the same length-k prefix and length-ℓ suffix and share the context $(a^{2m_0-k}, b^{2m_0-\ell})$, whereas $(a^{2m_0+1-k}, b^{2m_0+1-\ell}) \in \mathcal{D}_{L_{\pm,e}}(w_e) \setminus \mathcal{D}_{L_{\pm,e}}(w_c)$. The fixed-window counterexample criterion gives $L_{\pm,e} \notin \mathsf{KL}$. □

A concrete four-element typing. The regular-filter argument establishes the required existence of a finite typing. In fact, a four-element typing already suffices. Let $M_{\pm,e} := \{1, \kappa_{cd}, \kappa_e, 0\}$ be the monoid with identity 1 and $\mu\nu = 0$ for all $\mu, \nu \in \{\kappa_{cd}, \kappa_e, 0\}$, and define $s_{\pm,e} : \{a, b, c, d, e\}^+ \to M_{\pm,e}$ by

$$s_{\pm,e}(a) = s_{\pm,e}(b) = 1, \qquad s_{\pm,e}(c) = s_{\pm,e}(d) = \kappa_{cd}, \qquad s_{\pm,e}(e) = \kappa_e. \tag{3}$$

Then $L_{\pm,e}$ is $\sim_{s_{\pm,e}}$-substitutable. Indeed, two center-free nonempty factors sharing a context must coincide. The distribution of $a^i e b^j$ depends only on $i - j$, while that of $a^i \xi b^j$ for $\xi \in \{c, d\}$ depends only on $(i - j, \mathrm{par}(\xi) - i \bmod 2)$, where $\mathrm{par}(c) = 0$ and $\mathrm{par}(d) = 1$. A shared context forces these invariants to agree, and equal $s_{\pm,e}$-types distinguish e-factors from c/d-factors; thus equal-type factors with a common context have the same distribution.

Thus Theorem 8.3 applies to a nonregular linear language outside KL. The displayed grammar is even-linear, so no separation from Takada's control-set framework is claimed.

Finite-state control induced by yield typing. A single finite-state construction covers both the original typing s and its completion $\widehat{s}$. Let $t : \Sigma^* \to T$ be any finite-monoid morphism, and label each production of a linear grammar $G = (V, \Sigma, P, S)$ distinctly over $\mathsf{ProdLab}$. Write $A \Rightarrow_G^{\alpha} w$ for a derivation with production-label word α. For $\mathsf{Acc}_{\mathrm{ctrl}} \subseteq T$, put

$$\mathsf{Ctrl}_{t,\mathsf{Acc}_{\mathrm{ctrl}}}(G) := \{\alpha \in \mathsf{ProdLab}^* : S \Rightarrow_G^{\alpha} w \text{ for some } w \in \Sigma^*,\ t(w) \in \mathsf{Acc}_{\mathrm{ctrl}}\}.$$

This is regular: an automaton with states $V \times T \times T$ and an accepting state starts at $(S, 1_T, 1_T)$, updates $(A, \mu_L, \mu_R) \mapsto (B, \mu_L t(u), t(v)\mu_R)$ for each $A \to uBv$, and accepts a terminal rule $A \to w$ iff $\mu_L t(w) \mu_R \in \mathsf{Acc}_{\mathrm{ctrl}}$. The invariant is $(A, t(u), t(v))$ for a sentential form uAv; thus the controlled grammar generates $L(G) \cap t^{-1}(\mathsf{Acc}_{\mathrm{ctrl}})$.

The semigroup typing s is used through its monoid extension: taking $t = \widehat{s}$, start symbol A, and $\mathsf{Acc}_{\mathrm{ctrl}} = \{\mu\}$ selects precisely $L_G(A) \cap \widehat{s}^{-1}(\{\mu\})$, the fiber in Proposition 5.1, including the nullable type $\mathbf{1}_\sharp$. This is a finite-state bridge to Takada's regular control languages [11]; we do not claim arbitrary Takada control sets arise from yield typings.

9 Expressiveness and Boundaries

We now locate the framework between the fixed-window hierarchy and broader context-free families.

9.1 A nonlinear target inside RS

Put $\Delta := \{a^n b^n : n \geq 0\}$ and, for $z \in \{a, b\}^*$, let $\beta(z) := |z|_a - |z|_b$. The *height* after reading a prefix p is $\beta(p)$. Membership in Δ^* is characterized by the total height and the heights at the occurrences of ba:

$$w \in \Delta^* \iff \beta(w) = 0 \quad \text{and} \quad \beta(pb) = 0 \text{ whenever } w = pbap'. \tag{$\star$}$$

Indeed, a word in Δ^* is a product of blocks $a^n b^n$, whose nontrivial boundaries are precisely its occurrences of ba and have height zero. Conversely, cut at every occurrence of ba. The resulting factors contain no ba and therefore belong to $a^* b^*$; the right-hand conditions force each factor to start and end at height zero. Thus every factor is $a^n b^n$.

For a word x, define its *boundary-compatible entry-offset set* by

$$\mathcal{Q}(x) := \{\delta \in \mathbb{Z} : \delta + \beta(pb) = 0 \text{ for every factorization } x = pbap'\}.$$

These are formal integer offsets, not necessarily reachable nonnegative stack heights. If x contains no ba, then $\mathcal{Q}(x) = \mathbb{Z}$; otherwise $|\mathcal{Q}(x)| \leq 1$. For example, $\mathcal{Q}(ab) = \mathcal{Q}(bb) = \mathbb{Z}$, $\mathcal{Q}(ba) = \{1\}$, $\mathcal{Q}(aaba) = \{-1\}$, and $\mathcal{Q}(baaba) = \varnothing$.

For nonempty x, every occurrence of ba in uxv lies inside u, x, or v, or straddles one of the two concatenation boundaries. Hence $(\star)$ gives the following exact membership criterion:

$$\begin{aligned} uxv \in \Delta^* \iff \beta(uxv) = 0, \quad 0 \in \mathcal{Q}(u), \quad \beta(u) \in \mathcal{Q}(x), \quad \beta(ux) \in \mathcal{Q}(v), \\ \big(\mathrm{lst}(u), \mathrm{fst}(x)\big) = (b, a) \implies \beta(u) = 0, \\ \big(\mathrm{lst}(x), \mathrm{fst}(v)\big) = (b, a) \implies \beta(ux) = 0. \end{aligned} \tag{$\star\star$}$$

Here fst and lst denote the first and last letters of a nonempty word, with the value $\perp$ for the empty word. Thus membership of uxv depends on x only through $\beta(x)$, $\mathcal{Q}(x)$, and its first and last letters.

For nonempty w, let $s_\star(w)$ record its first and last letters and whether it contains an occurrence of ba. The summary of xy is determined by those of x and y: the outer letters are inherited, and a new ba can appear only across a boundary $b|a$. Thus $\ker s_\star$ is a semigroup congruence, and $s_\star$ is a surjective homomorphism onto its finite positive-word image, containing seven elements: the combination of first letter b, last letter a, and no occurrence of ba is impossible.

Proposition 9.1 (a nonlinear fixed-$s_\star$ target)**.** *The language Δ^* is deterministic context-free, nonregular, and nonlinear, and it is $\sim_{s_\star}$-substitutable. Moreover it is not (k,ℓ)-substitutable for any fixed $k, \ell \geq 0$. Hence $\Delta^* \in \mathsf{RS}_{s_\star}(\{a,b\}) \cap \mathrm{CFL}$ and $\Delta^* \in \mathsf{RS}(\{a,b\}) \setminus \mathsf{KL}(\{a,b\})$.*

Proof. A deterministic pushdown automaton recognizes Δ^* by tracking height, rejecting a b read at height zero and an a immediately after a b when the height is nonzero, and accepting at height zero; this follows from $(\star)$. The grammar $S \to TS \mid \lambda$, $T \to aTb \mid \lambda$ also generates Δ^*. Nonregularity follows from $\Delta^* \cap a^*b^* = \Delta$. For nonlinearity, linear languages are closed under regular intersection, whereas $\Delta^* \cap a^*b^*a^*b^* = \Delta\Delta$ is not linear [22, Section 6.1, before Theorem 6.1].

For substitutability, let nonempty x, y have equal $s_\star$-types and a common context (u, v). From $uxv, uyv \in \Delta^*$ and $(\star\star)$, we obtain $\beta(x) = \beta(y)$ and $\beta(u) \in \mathcal{Q}(x) \cap \mathcal{Q}(y)$. If neither x nor y contains ba, then $\mathcal{Q}(x) = \mathcal{Q}(y) = \mathbb{Z}$. Otherwise both contain ba, and each offset set has at most one element; hence $\mathcal{Q}(x) = \{\beta(u)\} = \mathcal{Q}(y)$. Since x, y also have equal first and last letters, $(\star\star)$ implies, for all u', v',

$$u'xv' \in \Delta^* \iff u'yv' \in \Delta^*.$$

Therefore $\mathcal{D}_{\Delta^*}(x) = \mathcal{D}_{\Delta^*}(y)$.

For the fixed-window separation, fix $k, \ell \geq 0$ and set $m_0 := \max\{1, k + \ell\}$, $x := a^{m_0}b^{m_0}$, and $y := a^{m_0}b^{m_0}a^{m_0}b^{m_0}$. The words share the same length-k prefix and length-ℓ suffix, and the empty context. But $axb \in \Delta^*$, while the central ba in ayb has height one immediately after its b; thus $ayb \notin \Delta^*$ by $(\star)$. The fixed-window counterexample criterion excludes every (k,ℓ). □

9.2 A capped regular counter family

Yoshinaka's regular language $L_{\mathrm{Yreg}} = ae^*ce^*a \cup ae^*de^*a \cup be^*ce^*b$ already gives a fixed-window separation [3, Proposition 1]; capped counters contrast with the uncapped obstruction below.

Let $\Sigma_c := \{\uparrow, \downarrow, \#\}$. For $w = c_1 \cdots c_n \in \Sigma_c^*$, let $\mathrm{ht}_i(w)$ be the height after i letters, with $\mathrm{ht}_0(w) = 0$, changing by $+1$ on $\uparrow$, -1 on $\downarrow$, and resetting to 0 on $\#$. For $B \geq 1$, put

$$\mathrm{CTR}_B := \{w \in \Sigma_c^* : 0 \leq \mathrm{ht}_i(w) \leq B \text{ for all } 0 \leq i \leq |w|\}.$$

For every $B \geq 1$, CTR_B is regular: a deterministic finite automaton records heights $0, \ldots, B$, uses one sink outside the range, and resets to 0 on $\#$. Hence $\mathrm{CTR}_B \in \mathsf{RS}(\Sigma_c)$ by Proposition 3.1.

Theorem 9.2. *For every $B \geq 2$, CTR_B is not (k,ℓ)-substitutable for any $k, \ell \geq 0$.*

Proof. Fix k, ℓ and put $x := (\uparrow\downarrow)^k \uparrow^{B-1} \#^\ell$ and $y := (\uparrow\downarrow)^k \uparrow^B \#^\ell$. They are long enough for the fixed-window criterion, have the same length-k prefix and length-ℓ suffix, and both lie in CTR_B, so they share the empty context. Yet $\uparrow x \in \mathrm{CTR}_B$ while $\uparrow y \notin \mathrm{CTR}_B$, the latter reaching height $B+1$. Hence their distributions differ, and the criterion applies. □

The bound $B \geq 2$ is sharp[2]. Let $h_B : \Sigma_c^* \to M_B$ be the DFA transition-monoid morphism recognizing CTR_B. Its positive restriction $s_B := h_B|_{\Sigma_c^+}$ is a finite-semigroup typing witnessing $\mathrm{CTR}_B \in \mathcal{C}^{\mathrm{cf}}_{s_B}$; only $B \geq 2$ lies outside the fixed-window hierarchy.

9.3 Natural deterministic context-free languages outside RS

Lemma 9.3 (finite-semigroup obstruction)**.** *Let $L \subseteq \Sigma^*$. Suppose an infinite $\Xi \subseteq \Sigma^+$ satisfies, for all distinct $x, y \in \Xi$, $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ and $\mathcal{D}_L(x) \neq \mathcal{D}_L(y)$. Then $L \notin \mathsf{RS}(\Sigma)$.*

Proof. Any finite s identifies two elements of Ξ; their common context and unequal distributions contradict $\sim_s$-substitutability. □

Removing the upper cap gives the deterministic context-free language

$$\mathrm{CTR} := \{w \in \Sigma_c^* : \mathrm{ht}_i(w) \geq 0 \text{ for all } 0 \leq i \leq |w|\},$$

accepted by the evident deterministic pushdown counter with reset.

Corollary 9.4 (uncapped counter obstruction)**.** $\mathrm{CTR} \notin \mathsf{RS}(\Sigma_c)$.

Proof. Use $\Xi = \{\uparrow^i : i \geq 1\}$. For $i < j$, $(\lambda, \downarrow^i)$ is common, while $(\lambda, \downarrow^j)$ is a context only of $\uparrow^j$. □

Define the one-bracket Dyck language by

$$D_1 := \{w \in \{a, b\}^* : |w'|_a \geq |w'|_b \text{ for every prefix } w',\ |w|_a = |w|_b\}.$$

It is a standard deterministic context-free language [23]. Here D_1 denotes the whole Dyck language, written D_1^* in the convention of Autebert–Berstel–Boasson [22].

Corollary 9.5. $D_1 \notin \mathsf{RS}(\{a, b\})$.

Proof. Use $\Xi = \{b^i a^i : i \geq 1\}$. For $i < j$, (a^j, b^j) is common, whereas (a^i, b^i) is a context only of $b^i a^i$; for $b^j a^j$ the prefix $a^i b^j$ underflows. □

Thus unbounded stack behavior alone is not the obstruction: the nonregular one-turn language $L_{\pm,e}$ lies in RS, but an infinite family of mutually comparable factors with distinct contextual behavior cannot be separated by any finite s.

Lemma 9.6 (fixed-word quotient)**.** *If L is $\sim_s$-substitutable and $z \in \Sigma^*$, then $L/z := \{w : wz \in L\}$ is $\sim_s$-substitutable.*

Proof. A common context (u, v) in L/z becomes (u, vz) in L; equality of distributions in L then transfers every context (u', v') by appending z. □

The language generated by $S \to aSS \mid b$ is the *Łukasiewicz language*. Yoshinaka [3, Section 3] already observed that this language is not (k, ℓ)-substitutable for any fixed k, ℓ. In the notation used here, $L_{\mathrm{Luk}} = D_1 b$ [22], and it is deterministic context-free [23]. The quotient argument gives the stronger conclusion that it lies outside the present finite-typing union: if $L_{\mathrm{Luk}} \in \mathsf{RS}(\{a, b\})$, then Lemma 9.6 would put $D_1 = L_{\mathrm{Luk}}/b$ in RS, contradicting Corollary 9.5.

[2] Indeed, CTR_1 is $(1,1)$-substitutable. Membership is characterized by the initial symbol not being $\downarrow$ and the absence of the length-2 factors $\uparrow\uparrow$, $\downarrow\downarrow$, and $\#\downarrow$. Thus, for nonempty factors occurring in words of CTR_1 with a common context, equality of their first and last symbols ensures equal membership under every context.

9.4 Comparison with Clark's congruential family

Clark [24] allows a finite initial set $I \subseteq V$; write $L_G[I] := \bigcup_{A \in I} L_G(A)$. Such a grammar is *congruential* if each $L_G(A)$ lies in one syntactic-congruence class of $L_G[I]$. Let $\mathsf{CONG}(\Gamma)$ be the languages over Γ generated by such grammars. Clark already places substitutable and (k,ℓ)-substitutable languages in this family; the following extends the containment to every fixed-s slice.

Proposition 9.7 (comparison with Clark's congruential family)**.** *For every finite alphabet Γ and finite-semigroup typing $s : \Gamma^+ \to \mathsf{S}$, $\mathcal{C}_s^{\mathrm{cf}} \subseteq \mathsf{CONG}(\Gamma)$. Consequently $\mathsf{RS}(\Gamma) \cap \mathrm{CFL} \subseteq \mathsf{CONG}(\Gamma)$, and for $\Gamma = \{a,b\}$ the inclusion is proper.*

Proof. For nonempty $L \in \mathcal{C}_s^{\mathrm{cf}}$, take any CFG for L and its trimmed $\mathsf{S}^\sharp$-typed refinement $\widetilde{G}$. The yields of each retained typed symbol A_μ have equal $\widehat{s}$-type and share a terminal reaching context. Lemma 2.2 implies that they lie in one syntactic-congruence class. This argument includes nullable symbols.

Delete the auxiliary $\widetilde{S}$ and use its typed successors $(S_0)_\mu$ as a finite initial set. The type $\mathbf{1}_\sharp$ already accounts for λ when it belongs to L; no extra initial symbol is necessary. A productionless initial symbol handles the empty language. Taking the union over all finite typings gives the inclusion.

For properness over $\{a,b\}$, use the Dyck example also discussed by Clark [24]. Every $x \in D_1$ satisfies $uxv \in D_1$ iff $uv \in D_1$, so $x \equiv_{D_1} \lambda$. Hence $S \to aSbS \mid \lambda$ is congruential, while Corollary 9.5 gives $D_1 \notin \mathsf{RS}(\{a,b\})$. □

10 Conclusion

Qualitative learnability is insensitive to grammatical normal forms, whereas quantitative data bounds reveal the algebraic structure of the typing. The exponential separation concerns typed thickness across BUNF presentations and characteristic samples for set-driven operators, not convergence of arbitrary Gold learners. The window–group hypothesis is sufficient, not claimed necessary; characterizing $\mathsf{RS} \cap \mathrm{CFL}$ remains open.

Data availability

No empirical data were used for the research described in this article.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Acknowledgments

The author thanks Makoto Kanazawa and Ryo Yoshinaka for guidance, encouragement, and help with relevant literature, and the two anonymous reviewers for careful and constructive reports that substantially improved the paper, in particular by identifying problems in the submitted

learning algorithm and by prompting the direct two-word witness in Theorem 9.2. The author also thanks Alexander Clark for valuable advice at ICGI 2014 during the early development of this work. Any remaining errors are the author's responsibility.

A Fixed-window witness bounds and thickness-preserving normalization

A.1 Proof of Lemma 7.1

Proof of Lemma 7.1. Put $m := k + \ell$, $q := |\mathsf{H}|$, $N := |V|$, and $\bar{\tau} := \tau_G + 1$. Fix $A \in V$, possibly S_0, and a retained typed copy A_μ with $\mu \neq \mathbf{1}_\sharp$; the copy $A_{\mathbf{1}_\sharp}$ yields only λ. Let η be the $(s_{k,\ell} \times g)$-value of some nonempty $w_0 \in L_G(A)$ of $\widehat{s}$-type μ. Among all derivation trees from A yielding a nonempty word of $(s_{k,\ell} \times g)$-value η, choose T, with yield w, minimizing first $|w|$ and then the number of nonterminal nodes. The kernel hypothesis gives $\widehat{s}(w) = \mu$, so $|\omega(A_\mu)| \leq |w|$ by Proposition 5.1. The bound is immediate if $|w| < m$; hence assume $|w| \geq m$.

The first k and last ℓ positions of w are disjoint. Protect them and choose one arbitrary position of w as a *nonemptiness anchor*, which may already belong to a protected window. Let Prot be the union of these positions. Then $1 \leq d := |\mathrm{Prot}| \leq m + 1$. In particular, when $m = 0$ the anchor still prevents the word from becoming empty. We use the following minimality consequence:

(∗) If w' arises by replacing pairwise disjoint factors of w whose occurrences contain no protected position, and $g(w') = g(w)$, then w' is nonempty and has the same $(s_{k,\ell} \times g)$-value η as w. Thus a derivation tree from A for such a w' cannot have shorter yield, or the same yield length and fewer nonterminal nodes, than T.

Indeed, the first k and last ℓ protected letters remain the corresponding windows, and $|w'| \geq d \geq 1$.

Consider only *nonterminal occurrences* in T, including those arising from unit and nullable productions. For each such node z, let $X_z \in V$ be its label, y_z its possibly empty terminal yield, and $P_z \subseteq \mathrm{Prot}$ the protected positions in its yield span. Call z *essential* if $P_z \neq \emptyset$ or $|y_z| > \bar{\tau}$.

Depth. If a proper descendant z' of z has the same triple $(P_z, X_z, g(y_z))$ as z, graft the subtree at z' in place of the subtree at z. The removed factors are disjoint from Prot, and the group value is preserved, so (∗) applies. The grafted tree has no longer yield and strictly fewer nonterminal nodes, a contradiction. Along any path the sets P_z are nested and take at most $d + 1$ values. Hence no path contains more than $(d+1)Nq \leq (m+2)Nq$ nonterminal nodes.

Width. If a level contained $d + q$ essential nodes, their disjoint yield spans would leave at least q nodes avoiding Prot. Choose q such nodes, in left-to-right order, with yields $a_1, \ldots, a_q$ of length greater than $\bar{\tau}$. For each, choose a word b_i generated by its nonterminal label with $|b_i| \leq \tau_G$; $b_i = \lambda$ is permitted. Write $w = c_0 a_1 c_1 \cdots a_q c_q$. For $1 \leq j \leq q$, put $r_j := g(a_1 c_1 \cdots a_j c_j)$ and $t_j := g(b_1 c_1 \cdots b_j c_j)$, and define $x_j := t_j r_j^{-1}$, with $r_0 = t_0 = x_0 := 1_\mathsf{H}$. Two of $x_0, \ldots, x_q$ coincide, say $x_i = x_j$ with $i < j$. Write the ordered interval products as $r_j = r_i r_{i:j}$ and $t_j = t_i t_{i:j}$. From $t_j r_i^{-1} = t_i t_{i:j} r_{i:j}^{-1} r_i^{-1}$, cancellation in this order gives $t_{i:j} = r_{i:j}$. Thus the corresponding interval words have the same g-value. Replace the corresponding pairwise disjoint subtrees by derivations of $b_{i+1}, \ldots, b_j$. This preserves the group value and all protected positions, but strictly shortens the yield, contradicting (∗). No commutativity of H is used. Therefore every level has at most $d + q - 1 \leq m + q$ essential nodes.

Counting. The essential nonterminal nodes are closed under ancestors and include the root, since $d \geq 1$. Their total number is at most $E := (m+q)(m+2)Nq$. As G is in BUNF, at most $2E$ maximal nonessential subtrees hang below them, each yielding at most $\bar{\tau}$ letters. Every remaining terminal letter is emitted by an essential lexical node, hence is protected, because a

lexical node has yield length $1 \le \bar{\tau}$. There are at most d such letters. Thus

$$|w| \le d + 2E\bar{\tau} \le m + 1 + [1 + 2(m+q)(m+2)Nq](\tau_G + 1). \qquad \square$$

A.2 Thickness-preserving BUNF normalization

Proof of Proposition 7.3. Let G_* be reduced and put $n := |G_*|$, $\bar{\tau} := \tau_{G_*} + 1$. First isolate terminals in right-hand sides of length at least two, using wrappers $W_a \to a$, and then binarize. Retain all nullable and unit rules. These standard operations preserve all original nonterminal languages [22, Section 3.1] [25, Sections 7.1 and 7.4.2]. Each auxiliary binarization symbol is fresh for its original production and represents a contiguous block of that production's right-hand side, containing at most $O(n)$ original symbols. Substituting shortest yields gives a terminal word of length $O(n\bar{\tau})$. The original nonterminal languages and their shortest yields are unchanged. Trim unreachable or unproductive symbols. The resulting BUNF grammar is equivalent, has polynomial size and $\tau_G = O(n(\tau_{G_*} + 1))$. $\square$

B Proof of the linear-spine BUNF normalization

Proof of Proposition 8.1. A linear CFG has rules $A \to uBv$ or $A \to w$, for terminal words $u, v, w \in \Sigma^*$. Retain $A \to B$ and $A \to \lambda$. No nullable or unit elimination is needed.

For each $a \in \Sigma$ introduce a fresh terminal wrapper $W_a \to a$, and choose all other auxiliary spine symbols freshly per production. For a rule $A \to a_1 \cdots a_m B b_1 \cdots b_n$ with $m + n > 0$, form the chains

$$\begin{aligned} C_0^{\mathrm{R}} &:= B, \quad & C_j^{\mathrm{R}} &\to C_{j-1}^{\mathrm{R}} W_{b_j} \qquad (1 \le j \le n), \\ C_0^{\mathrm{L}} &:= C_n^{\mathrm{R}}, \quad & C_i^{\mathrm{L}} &\to W_{a_{m-i+1}} C_{i-1}^{\mathrm{L}} \quad (1 \le i \le m). \end{aligned}$$

Identify $C_n^{\mathrm{R}} = A$ when $m = 0$, and $C_m^{\mathrm{L}} = A$ otherwise. For terminal-only $A \to a_1 \cdots a_k$, keep $k = 0, 1$ unchanged. If $k \ge 2$, let $C_1^{\mathrm{T}} = A$ and use fresh $C_2^{\mathrm{T}}, \ldots, C_k^{\mathrm{T}}$ with

$$C_i^{\mathrm{T}} \to W_{a_i} C_{i+1}^{\mathrm{T}} \ (1 \le i < k), \qquad C_k^{\mathrm{T}} \to a_k.$$

Every new binary rule has one wrapper child producing exactly one terminal, and one continuing spine child. The original languages are preserved. Trim to obtain a reduced BUNF grammar of polynomial encoded size. $\square$

C Substitutability of the bracketed doubling family

Put $L_0 = \{\mathtt{a}\}$ and $L_i = \lceil L_{i-1} \rceil \cup \lfloor L_{i-1} L_{i-1} \rfloor$ for $i \ge 1$; thus $L_{G_n}(E_i) = L_i$ for $0 \le i \le n$, and $L_n = L(G_n)$. Fix $n \ge 1$. We prove Proposition 6.4(i) with a deterministic top-down parser for G_n whose transitions are injective for each fixed input letter.

The parser. A *stack* is a word over $E_0, \ldots, E_n, \rceil, \rfloor$, with its top on the left; λ is the empty stack. For every stack ψ and $1 \le i \le n$ the moves are

$$\begin{aligned} &E_0\psi \xrightarrow{\mathtt{a}} \psi, \quad E_i\psi \xrightarrow{\lceil} E_{i-1}\rceil\psi, \quad E_i\psi \xrightarrow{\lfloor} E_{i-1}E_{i-1}\rfloor\psi, \\ &\rceil\psi \xrightarrow{\rceil} \psi, \qquad \rfloor\psi \xrightarrow{\rfloor} \psi, \end{aligned}$$

and there are no others; in particular, no move leaves λ. Write $\sigma \xrightarrow{w} \theta$ if reading w from σ ends in θ. The parser starts with the stack E_n and accepts w if $E_n \xrightarrow{w} \lambda$; Lemma C.1 shows that it accepts exactly L_n. We use three properties.

(P1) *Determinism and letterwise injectivity.* For each input letter c, the transition relation defines a partial injective map on stacks: there is at most one c-successor of each stack, and two c-transitions with the same target have the same source. For opening brackets, the output reveals the expanded E_i and the untouched suffix; for $\mathtt{a}$, $\rceil$, and $\lfloor$, the input letter identifies the popped top symbol. Consequently every word acts as a partial injection on stacks: if $\sigma \xrightarrow{w} \theta$ and $\sigma \xrightarrow{w} \theta'$, then $\theta = \theta'$; if $\sigma \xrightarrow{w} \theta$ and $\sigma' \xrightarrow{w} \theta$, then $\sigma = \sigma'$.

(P2) *Weight.* Give E_i weight 4^i and each closing marker weight 1, and let $\mathrm{wt}(\sigma)$ be the total weight of a stack σ. Every move lowers wt: each pop removes weight 1, and $4^i > 2 \cdot 4^{i-1} + 1$ for $i \geq 1$. Hence $\sigma \xrightarrow{w} \sigma$ only for $w = \lambda$.

(P3) *Locality.* Moves inspect and rewrite only the top symbol, and closing markers are only popped. Hence $\sigma \xrightarrow{w} \theta$ implies $\sigma\psi \xrightarrow{w} \theta\psi$. Conversely, let $\sigma_1 \neq \lambda$ and $\sigma_1\psi \xrightarrow{w} \theta$. Then either

- (a) the run visits ψ: splitting at the first visit gives $w = w_1w_2$ with $\sigma_1 \xrightarrow{w_1} \lambda$ and $\psi \xrightarrow{w_2} \theta$, so $\mathrm{wt}(\theta) \leq \mathrm{wt}(\psi)$; or
- (b) it does not: $\theta = \theta_1\psi$ with $\sigma_1 \xrightarrow{w} \theta_1 \neq \lambda$, so $\mathrm{wt}(\theta) > \mathrm{wt}(\psi)$, and θ_1 ends with the last symbol of σ_1 whenever that symbol is a closing marker.

Thus, for the fixed suffix ψ, the comparison of $\mathrm{wt}(\theta)$ with $\mathrm{wt}(\psi)$ determines which case occurs.

Lemma C.1. *For $0 \leq i \leq n$ and every stack ψ, $E_i\psi \xrightarrow{w} \psi$ if and only if $w \in L_i$; in particular, the parser accepts exactly L_n. Moreover, if $\sigma \xrightarrow{w} \theta$ and $w \in L_i$, then $\sigma = E_i\theta$.*

Proof. A run $E_i\psi \xrightarrow{w} \psi$ falls under case (a) of (P3) with $\sigma_1 = E_i$, since case (b) would end at $\theta_1\psi \neq \psi$. So $w = w_1w_2$ with $E_i \xrightarrow{w_1} \lambda$ and $\psi \xrightarrow{w_2} \psi$, and $w_2 = \lambda$ by (P2). Together with the lifting in (P3), it suffices to show that $E_i \xrightarrow{w} \lambda$ iff $w \in L_i$. We induct on i. From E_0 only the $\mathtt{a}$-move is possible, and λ admits no further move. For $i \geq 1$, the first move reads $\lceil$ or $\lfloor$. After $\lceil$, the stack $E_{i-1}\rceil$ must be emptied; as its target λ excludes case (b), (P3) with $\sigma_1 = E_{i-1}$ and $\psi = \rceil$ splits the remaining input as zz' with $E_{i-1} \xrightarrow{z} \lambda$ and $\rceil \xrightarrow{z'} \lambda$, whence $z' = \rceil$. After $\lfloor$, two such splittings of $E_{i-1}E_{i-1}\lfloor$ give $z_1z_2\lfloor$ with $E_{i-1} \xrightarrow{z_j} \lambda$. By induction, $E_i \xrightarrow{w} \lambda$ iff $w \in \lceil L_{i-1}\rceil \cup \lfloor L_{i-1}L_{i-1}\rfloor = L_i$, the required runs being obtained by lifting.

For the last claim, the first part gives $E_i\theta \xrightarrow{w} \theta$, and (P1) gives $\sigma = E_i\theta$. □

Lemma C.2. *If $0 \leq i \leq n$ and $w, w' \in L_i$, then $w \equiv_{L_n} w'$.*

Proof. If $pwq \in L_n$, Lemma C.1 gives a run $E_n \xrightarrow{p} \sigma \xrightarrow{w} \theta \xrightarrow{q} \lambda$ with $\sigma = E_i\theta$. The same lemma gives $E_i\theta \xrightarrow{w'} \theta$, so $pw'q \in L_n$. Exchanging w and w' proves the converse. □

Lemma C.3 (branch synchronization)**.** *Let $\sigma \xrightarrow{x} \theta$ and $\sigma \xrightarrow{y} \theta$, where x, y are nonempty and begin with different letters. Then $\sigma = E_i\psi$ for some $1 \leq i \leq n$ and some stack ψ, and $x = wx'$, $y = w'y'$ with $w, w' \in L_i$, $\psi \xrightarrow{x'} \theta$ and $\psi \xrightarrow{y'} \theta$.*

Proof. Only a top E_i with $i \geq 1$ admits two different letters, namely $\lceil$ and $\lfloor$. Say $x = \lceil x_0$ and $y = \lfloor y_0$, so $E_{i-1}\rceil\psi \xrightarrow{x_0} \theta$ and $E_{i-1}E_{i-1}\lfloor\psi \xrightarrow{y_0} \theta$. Apply (P3) to both runs with this ψ. Since the comparison of $\mathrm{wt}(\theta)$ with $\mathrm{wt}(\psi)$ decides the case, both runs are in the same case. Case (b) is impossible: $\theta = \theta_1\psi = \theta_2\psi$ would give $\theta_1 = \theta_2$, although θ_1 ends with $\rceil$ and θ_2 with $\lfloor$. In case (a), $x_0 = x_1x'$ with $E_{i-1}\rceil \xrightarrow{x_1} \lambda$ and $\psi \xrightarrow{x'} \theta$. Thus $w := \lceil x_1$ satisfies $E_i \xrightarrow{w} \lambda$, and $w \in L_i$ by Lemma C.1. The same argument gives $y = w'y'$. □

Proposition C.4. *If $\sigma \xrightarrow{x} \theta$ and $\sigma \xrightarrow{y} \theta$, then $x \equiv_{L_n} y$.*

Proof. Induct on $|x|+|y|$. If x or y is empty, then $\sigma = \theta$, and the other is empty by (P2). If $x = cx'$ and $y = cy'$ for a letter c, determinism gives $\sigma \xrightarrow{c} \sigma'$ with $\sigma' \xrightarrow{x'} \theta$ and $\sigma' \xrightarrow{y'} \theta$, so $x' \equiv_{L_n} y'$ by induction and hence $x \equiv_{L_n} y$. Otherwise Lemma C.3 gives $x = wx'$ and $y = w'y'$ as there. Since w, w' are nonempty, induction gives $x' \equiv_{L_n} y'$, Lemma C.2 gives $w \equiv_{L_n} w'$, and $\equiv_{L_n}$ is a monoid congruence. □

Proof of Proposition 6.4(i). The finite language L_n is context-free. Let $x, y \in \Gamma^+_{\mathrm{br}}$ and $(u, v) \in \mathcal{D}_{L_n}(x) \cap \mathcal{D}_{L_n}(y)$. By Lemma C.1 there are runs $E_n \xrightarrow{u} \sigma \xrightarrow{x} \theta \xrightarrow{v} \lambda$ and $E_n \xrightarrow{u} \sigma' \xrightarrow{y} \theta' \xrightarrow{v} \lambda$. By (P1), $\sigma = \sigma'$ and $\theta = \theta'$, so Proposition C.4 gives $x \equiv_{L_n} y$, that is, $\mathcal{D}_{L_n}(x) = \mathcal{D}_{L_n}(y)$. Thus L_n is Clark–Eyraud substitutable. Adding the premise $s(x) = s(y)$ only weakens this condition, so L_n is $\sim_s$-substitutable for every s; in particular $L_n \in \mathcal{C}^{\mathrm{cf}}_{\mathrm{s0}} \cap \mathcal{C}^{\mathrm{cf}}_{\mathrm{sv}}$. □

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation and revision of this manuscript, the author used OpenAI's ChatGPT and Anthropic's Claude to assist with language editing, manuscript organization, and the examination of mathematical arguments. The author reviewed and edited the AI-assisted content and takes full responsibility for the results and proofs presented in this article.

References

[1] E. M. Gold. Language identification in the limit. *Information and Control*, 10(5):447–474, 1967.

[2] A. Clark and R. Eyraud. Polynomial identification in the limit of substitutable context-free languages. *Journal of Machine Learning Research*, 8:1725–1745, 2007.

[3] R. Yoshinaka. Identification in the limit of k,l-substitutable context-free languages. In *Grammatical Inference: Algorithms and Applications – ICGI 2008*, LNCS 5278, pp. 266–279, Springer, 2008. doi:10.1007/978-3-540-88009-7_21.

[4] C. de la Higuera. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, 2010.

[5] M. Kanazawa. Identification in the limit of categorial grammars. *Journal of Logic, Language and Information*, 5(2):115–155, 1996. doi:10.1007/BF00173697.

[6] M. Kanazawa. *Learnable Classes of Categorial Grammars*. CSLI Publications, 1998.

[7] T. Kuriyama. *Learning Algorithm for Relation-Substitutable Context-Free Languages*. arXiv:1409.6247v3, November 2014 (historical version). `https://arxiv.org/abs/1409.6247v3`.

[8] R. Yoshinaka. General perspective on distributionally learnable classes. In *Proceedings of the 14th Meeting on the Mathematics of Language (MoL 14)*, pp. 87–98, 2015.

[9] F. Coste, D. Fredouille, C. Kermorvant, and C. de la Higuera. Introducing domain and typing bias in automata inference. In *ICGI 2004*, LNCS 3264, pp. 115–126, Springer, 2004. doi:10.1007/978-3-540-30195-0_11.

[10] Y. Takada. Grammatical inference for even linear languages based on control sets. *Information Processing Letters*, 28(4):193–199, 1988. doi:10.1016/0020-0190(88)90208-6.

[11] Y. Takada. Learning formal languages based on control sets. In K. P. Jantke and S. Lange (Eds.), *Algorithmic Learning for Knowledge-Based Systems*, LNAI 961, pp. 316–339, Springer, 1995. doi:10.1007/3-540-60217-8_15.

[12] S. Eilenberg. *Automata, Languages, and Machines*, Vol. A. Academic Press, 1974.

[13] M. Wakatsuki and E. Tomita. A fast algorithm for checking the inclusion for very simple deterministic pushdown automata. *IEICE Transactions on Information and Systems*, E76-D(10):1224–1233, 1993.

[14] R. Eyraud, J. Heinz, and R. Yoshinaka. Efficiency in the identification in the limit learning paradigm. In J. Heinz and J. M. Sempere (Eds.), *Topics in Grammatical Inference*, pp. 25–46, Springer, 2016. doi:10.1007/978-3-662-48395-4_2.

[15] C. de la Higuera. Characteristic sets for polynomial grammatical inference. *Machine Learning*, 27:125–138, 1997.

[16] A. Clark. Learning trees from strings: A strong learning algorithm for some context-free grammars. *Journal of Machine Learning Research*, 14:3537–3559, 2013.

[17] J.-E. Pin. *Varieties of Formal Languages*. Translated by A. Howie. Plenum Press, New York, 1986. ISBN 0-306-42294-8.

[18] J.-E. Pin. Syntactic semigroups. In G. Rozenberg and A. Salomaa (eds.), *Handbook of Formal Languages*, Vol. 1, pp. 679–746, Springer, 1997.

[19] J. C. Costa. Some pseudovariety joins involving locally trivial semigroups. *Semigroup Forum*, 64(1): 12–28, 2002. doi:10.1007/ s002330010060.

[20] J. C. Costa. Some pseudovariety joins involving groups and locally trivial semigroups. In *Semigroups, Algorithms, Automata and Languages*, pp. 341–348, World Scientific, 2002. doi:10.1142/9789812776884_0013.

[21] S. Ginsburg and E. H. Spanier. Finite-turn pushdown automata. *SIAM Journal on Control*, 4(3):429–453, 1966. doi:10.1137/0304034.

[22] J.-M. Autebert, J. Berstel, and L. Boasson. Context-free languages and pushdown automata. In G. Rozenberg and A. Salomaa (Eds.), *Handbook of Formal Languages*, Vol. 1, pp. 111–174, Springer, 1997.

[23] S. Ginsburg and S. A. Greibach. Deterministic context-free languages. *Information and Control*, 9: 620–648, 1966.

[24] A. Clark. Distributional learning of some context-free languages with a minimally adequate teacher. In *Proceedings of ICGI 2010*, LNCS 6339, pp. 24–37, Springer, 2010. doi:10.1007/978-3-642-15488-1_4.

[25] J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed., Pearson/Addison-Wesley, 2007.

Declaration of Competing Interests

Manuscript ID: TCS-D-26-00494
Title: Distributional Learning of Context-Free Languages under Fixed Finite-Semigroup Typing
Author: Takayuki Kuriyama

The author declares no known competing financial interests or personal relationships that could have influenced the work reported in this paper.